DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim

Student Name: Trapti Kulshrestha

STUDENT ID: 019124833

Homework: 3

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Air M4

Local Model: qwen3:8b

Tagged Commit Hash:

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Part 1 = FastAPI authentication + sessions + Bootstrap

Section 1: Authentication Application

1.1 Folder Structure:

For Homework 3, I continued working in the same data260-4833 repository instead of making a separate project. I added a routers/ folder for authentication related routes & a templates/ folder for the HTML pages.

The authentication logic is kept inside routers/auth.py using FastAPI APIRouter. This helped me keep login related code separate from the existing Clinical Trial Listings code, while everything still remains inside the same repository.

1.2 Session Middleware and Router Integration:

I integrated the HW3 authentication router into my existing FastAPI application instead of creating a separate app. I added SessionMiddleware to support user sessions and registered the authentication routes using app.include_router(auth_router). Since the previous homework already used / for the Clinical Trial frontend, I moved that route to /trials so the new authentication home page could use / without removing the earlier functionality.

While integrating the authentication starter code into my existing project, I faced a few compatibility issues. First, the required session and template dependencies were not installed in my virtual environment, which caused errors for itsdangerous and jinja2. After installing the missing packages, the application started successfully. I also encountered a TemplateResponse compatibility issue because the starter code used an older calling style. I updated the template calls to use explicit request, name, and context arguments, which resolved the error and allowed the authentication pages to render correctly.

main.py M

main.py > home

```python
from fastapi import FastAPI, HTTPException, Query
from fastapi.responses import FileResponse
from pydantic import BaseModel, EmailStr
from typing import List, Optional
import uvicorn
import os
from starlette.middleware.sessions import SessionMiddleware  #extension of hw3
from routers.auth import router as auth_router


# I am creating one FastAPI application for my Clinical Trial Listings domain.
app = FastAPI(
    title="Clinical Trial Listings API",
    version="2.0.0"
)

SECRET_KEY = os.getenv("SECRET_KEY", "dev-only-secret-key")

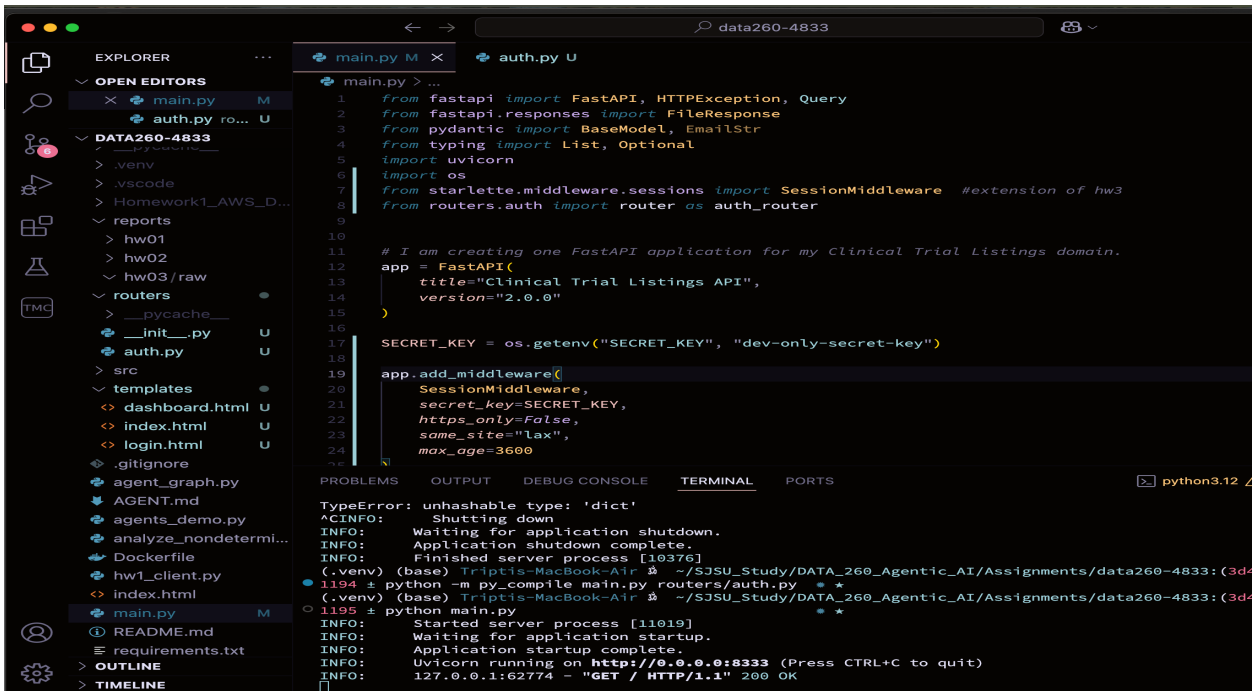
app.add_middleware(
    SessionMiddleware,
    secret_key=SECRET_KEY,
    https_only=False,
    same_site="lax",
    max_age=3600
)

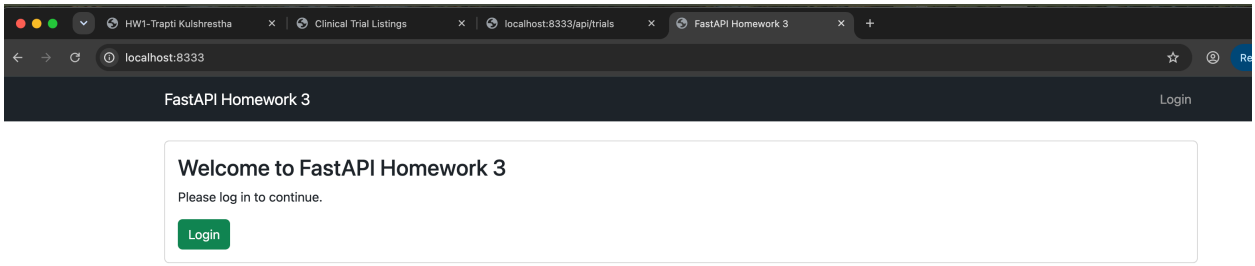
app.include_router(auth_router)

# ------------------------------------------------------------
```

1.3 Home Route:

The / route now serves as the authentication home page for Homework 3. When the user is not logged in, the page displays a simple Bootstrap interface with a login option. This route is handled from routers/auth.py, while my previous Clinical Trial application is still preserved under the /trials route so earlier functionality is not lost.





I also customized the starter Bootstrap interface to better match my assigned domain, Clinical Trial Listings. I replaced the generic Homework 3 branding with domain-specific text, added a “Secure Research Portal” label, improved the spacing and card styling, and used a cleaner background and button layout. These changes make the application look more polished while keeping the interface simple and easy to use.

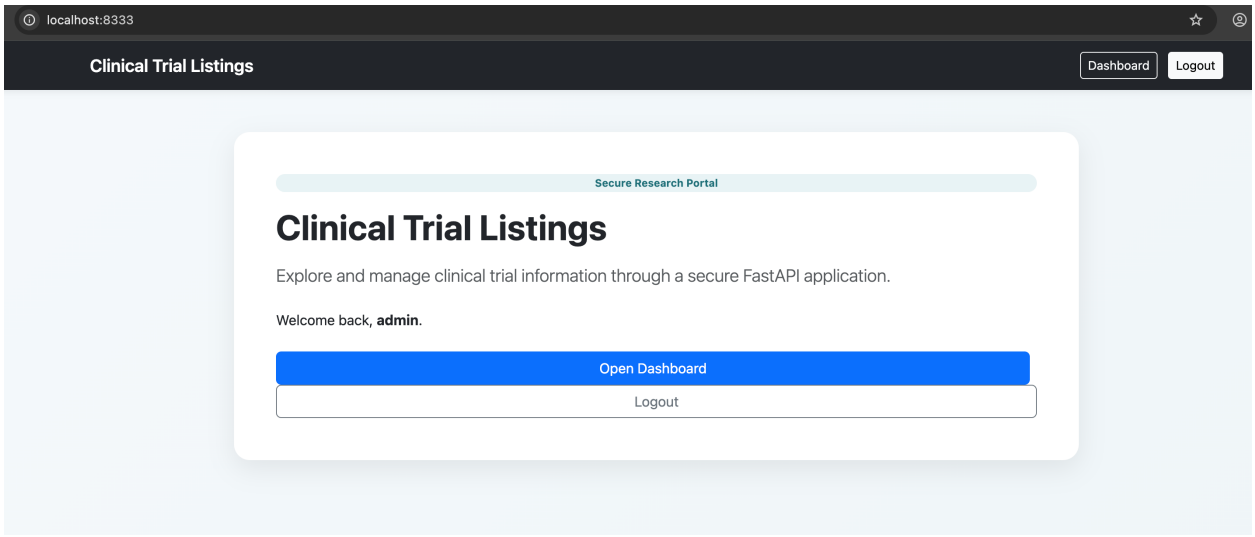
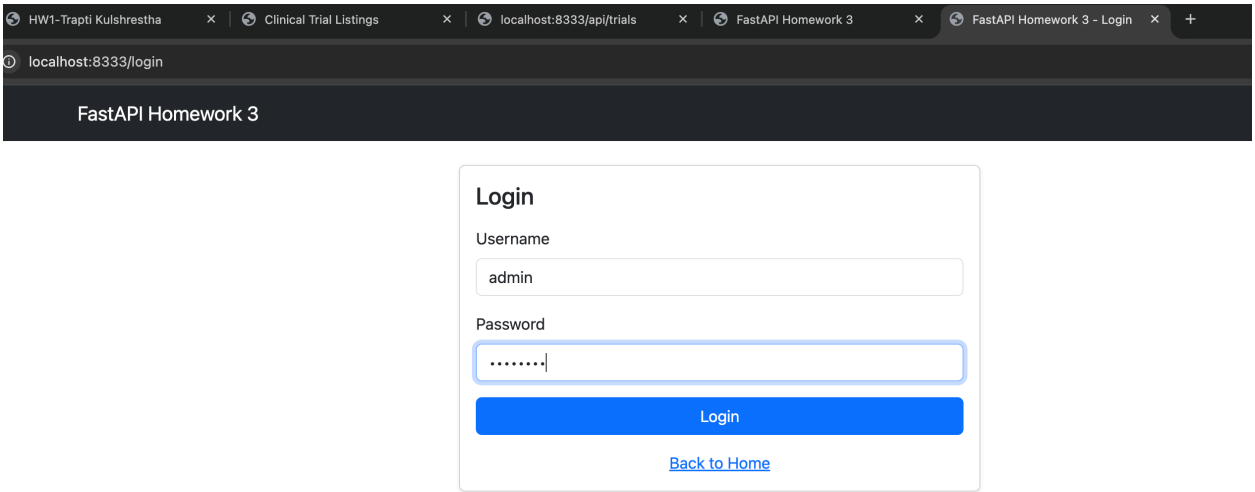
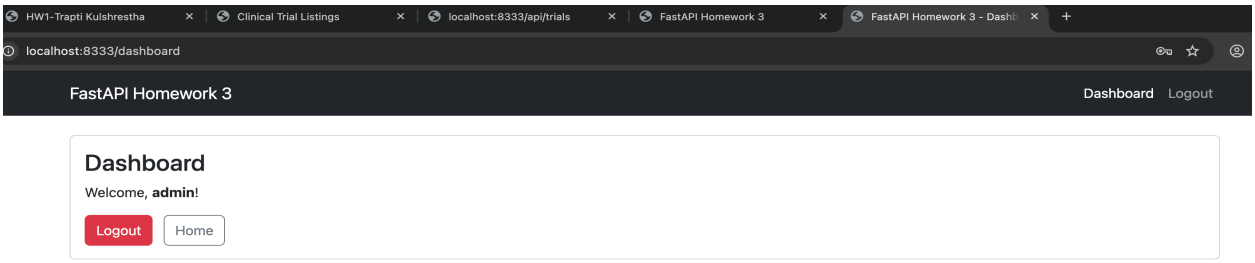


Figure: Updated Clinical Trial Listings home page with domain-specific Bootstrap styling.

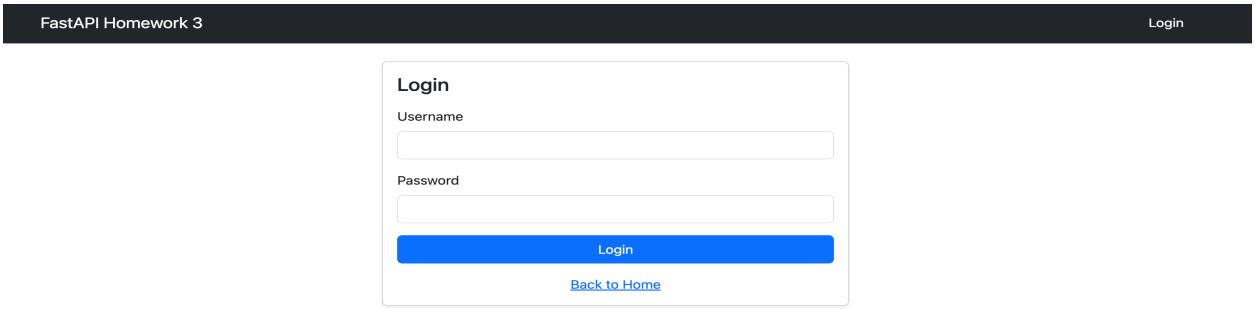
1.4 Login Route and Invalid Login Handling:

The /login route provides a Bootstrap login form for username and password authentication. I tested both successful and unsuccessful login attempts. For invalid credentials, I modified the starter implementation so that the page displays a Bootstrap danger alert instead of silently redirecting back to the login page. This gives the user clear feedback while keeping them on the login screen.





With wrong password, currently, not showing any error, went back to home page with starter code.



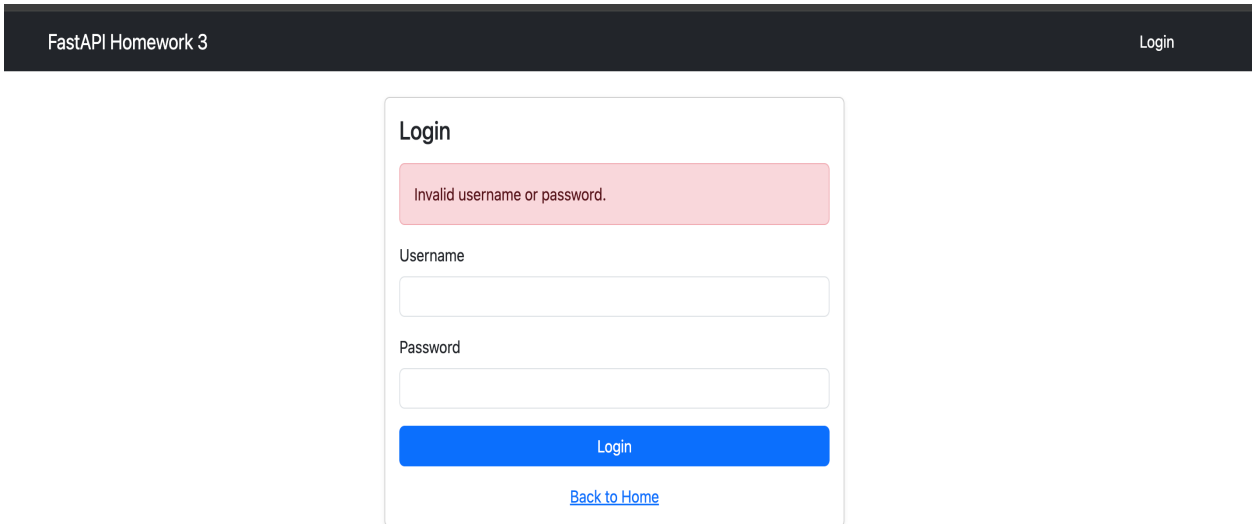

```
        # Redirect back to login page
        #
        # NOTE:
        # No error message is shown intentionally.
        # Students are expected to add Bootstrap alerts.
        return templates.TemplateResponse(  #replaced this section...hw3
        request=request,
        name="login.html",
        context={
            "user": None,
            "error": "Invalid username or password."
        },
        status_code=401
    )


@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected route.

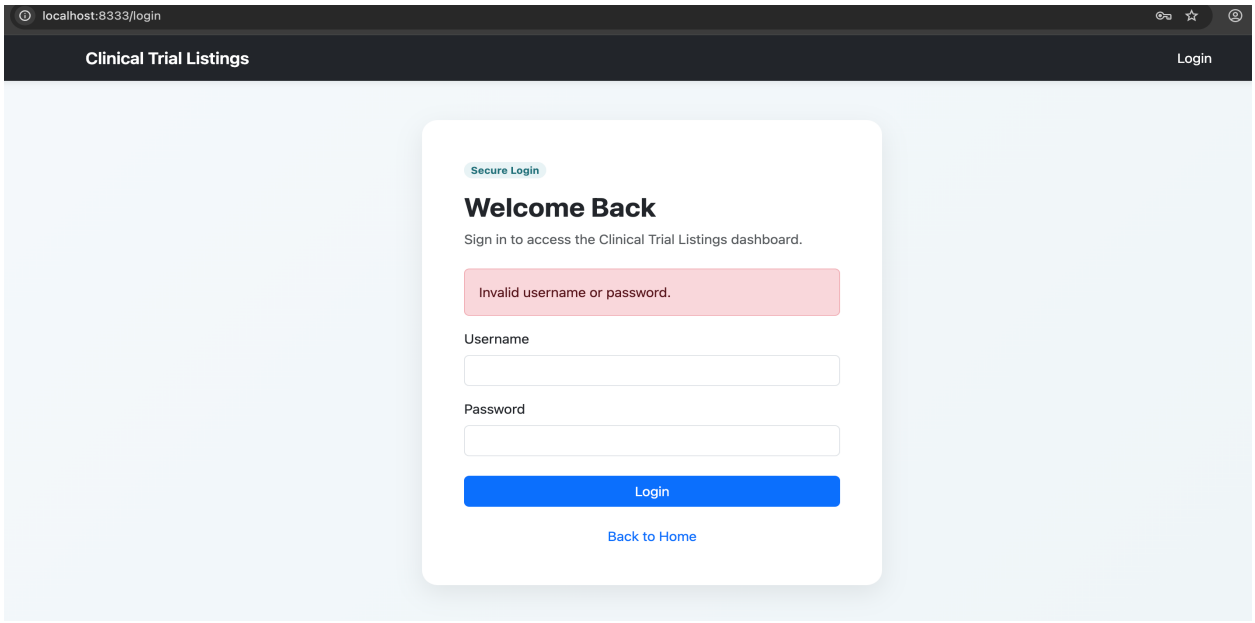
    - Only accessible if user is logged in
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  python3.12
INFO:     127.0.0.1:65077 - "GET /logout HTTP/1.1" 302 Found
```

```
29          <div class="card-body">
30            <h4 class="card-title mb-3">Login</h4>
31
32            <!-- Intentionally no alert shown for invalid login.
33                 Students can add a Bootstrap Alert as an extra step.
34                 -->
35            {% if error %}
36            <div class="alert alert-danger" role="alert">
37            {{ error }}
38            </div>
39            {% endif %}
40
41            <form method="post" action="/login">
42              <div class="mb-3">
43                <label class="form-label">Username</label>
44                <input class="form-control" type="text" name="username
45              </div>
46
47              <div class="mb-3">
48                <label class="form-label">Password</label>
49                <input class="form-control" type="password" name="pass
```



Updated_version:



In the POST /login route, valid credentials are stored in the session using request.session["user"], and the user is redirected to /dashboard. If the credentials are incorrect, the same login template is returned with an error value in the template context. The HTML checks for this value and displays the Bootstrap alert only when an error exists.

1.5 Protected Dashboard:

After a successful login, the user is redirected to the protected /dashboard route. The dashboard reads the username stored in the session and displays a personalized welcome message. If a user tries to open /dashboard without a valid session, the application redirects them back to /login, which prevents unauthorized access to the protected page.

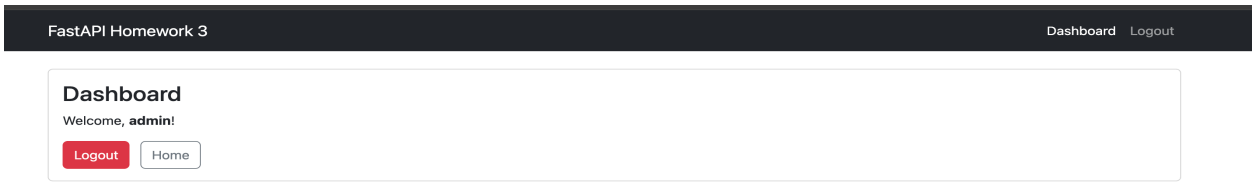
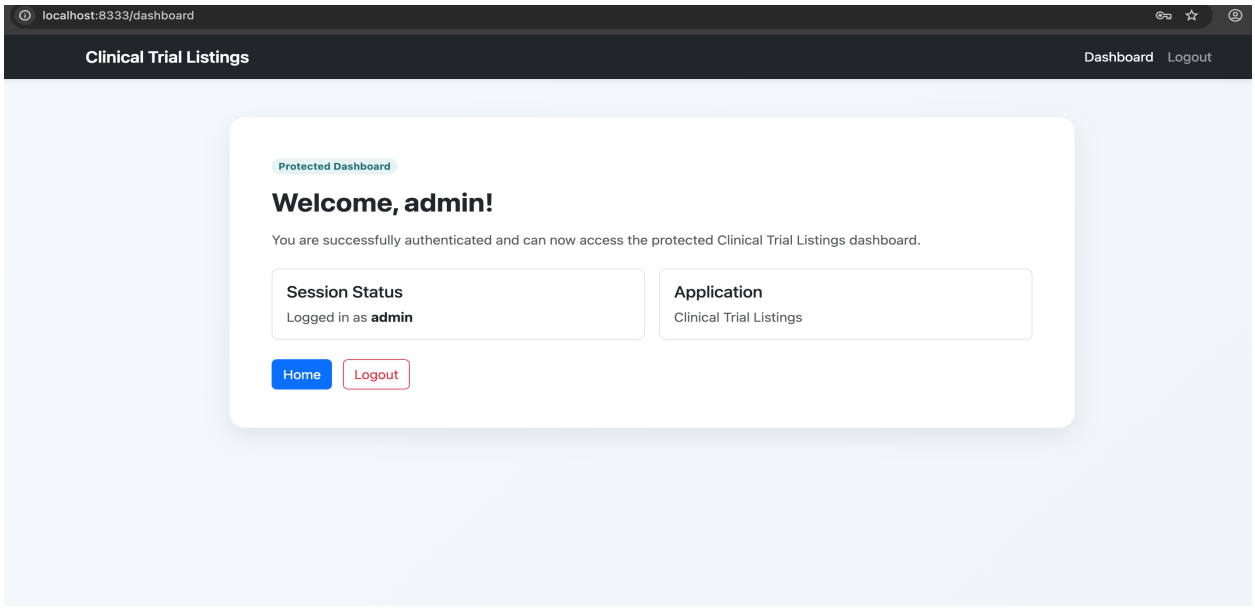


Figure: Protected dashboard displayed after successful login with the authenticated user stored in the session.

I also updated the dashboard styling so it matches the Clinical Trial Listings theme used on the home and login pages. The dashboard now clearly shows the authenticated username, session status, and application name while keeping the logout action visible.



1.6 Logout:

The /logout route clears the active user session and redirects the user back to the home page. I verified that after logging out, manually revisiting /dashboard no longer gives access to the protected page. Instead, the application redirects the user to /login, confirming that the session was successfully destroyed.

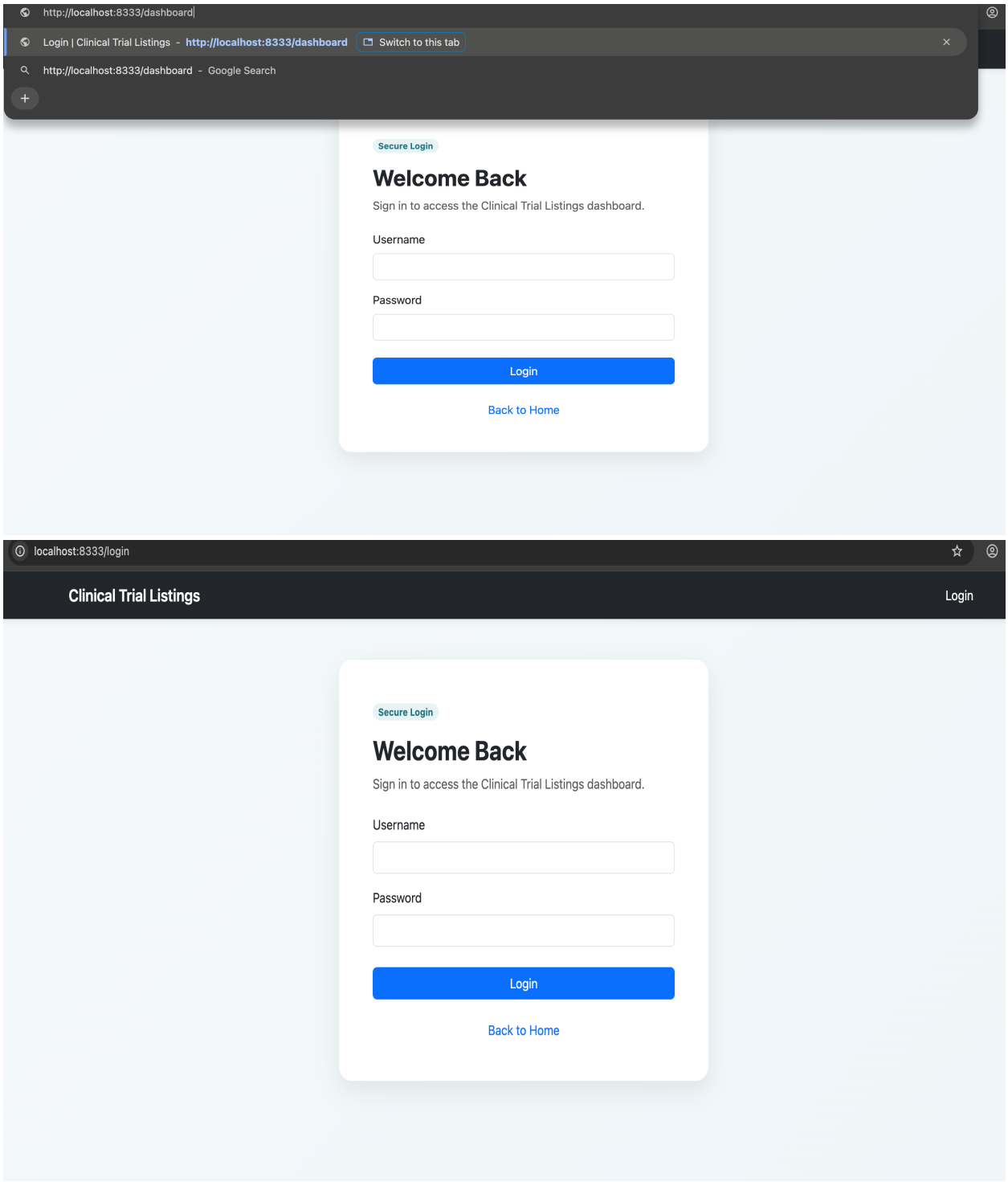
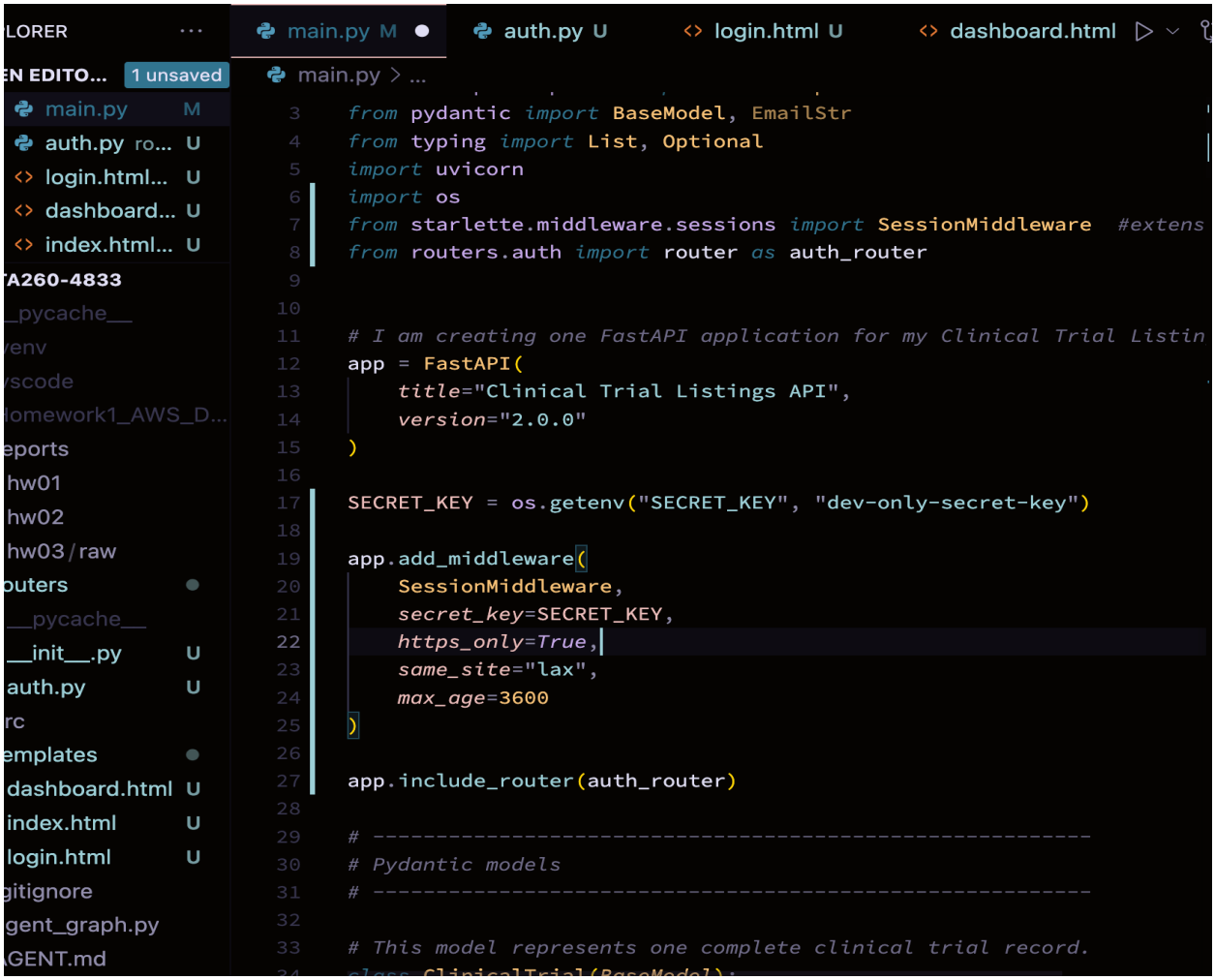


Figure: Attempting to access /dashboard after logout redirects the user to the login page

1.7 Session Cookie Security:

I configured SessionMiddleware so that the authentication session cookie includes the Secure, HttpOnly, and SameSite=Lax attributes. I verified the settings by sending a login request from the terminal and inspecting the Set-Cookie response header. The response showed httponly, samesite=lax, and secure, confirming that the required cookie protections were applied. The header also showed Max-Age=3600, meaning the session cookie is valid for one hour unless it is cleared earlier.



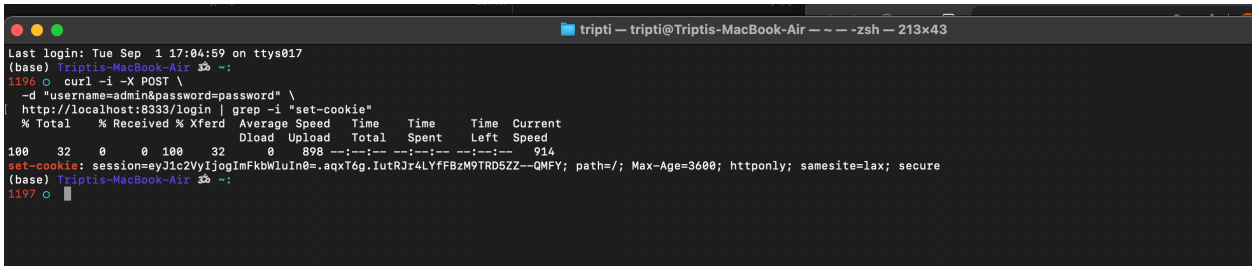
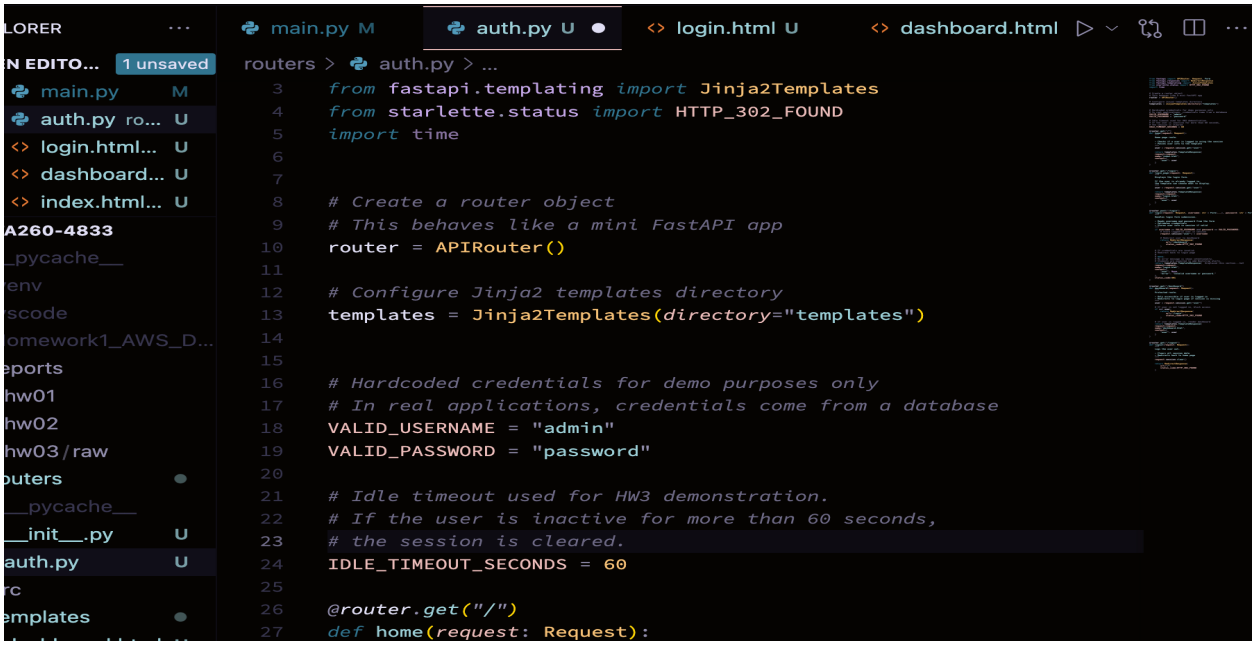
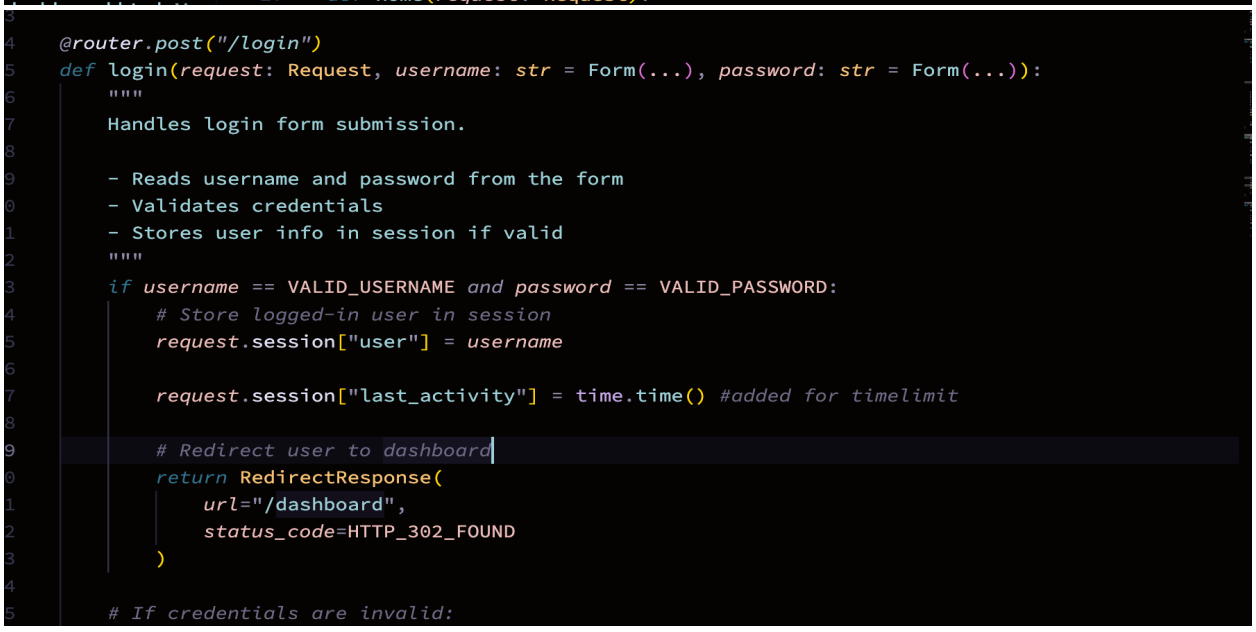


Figure: Set-Cookie response header showing Secure, HttpOnly, SameSite=Lax, and Max-Age=3600

1.8 Idle Session Timeout:

I implemented an idle-session timeout so that an authenticated session is cleared after a period of inactivity. When the user logs in successfully, the application stores the current time in the session as last_activity. Each time the protected dashboard is accessed, the application compares the current time with the stored activity time. For testing, I used a 60-second idle timeout. If the user remains inactive for longer than this limit, the session is cleared and the user is redirected to the login page with a warning message. I verified that the expired session could no longer be reused to access the protected dashboard.





```python
@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected dashboard route.

    The user must have a valid session.
    The session also expires after a period of inactivity.
    """

    user = request.session.get("user")
    last_activity = request.session.get("last_activity")

    # No authenticated user -> redirect to login
    if not user:
        return RedirectResponse(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # Check whether the session has been idle too long
    if last_activity is not None:

        idle_time = time.time() - last_activity

        if idle_time > IDLE_TIMEOUT_SECONDS:

            # Remove all session data after timeout
            request.session.clear()

            return RedirectResponse(
```

```python
@router.get("/login")
def login_page(request: Request):

    user = request.session.get("user")

    expired = request.query_params.get("expired")

    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": user,
            "expired": expired
        }
    )
```

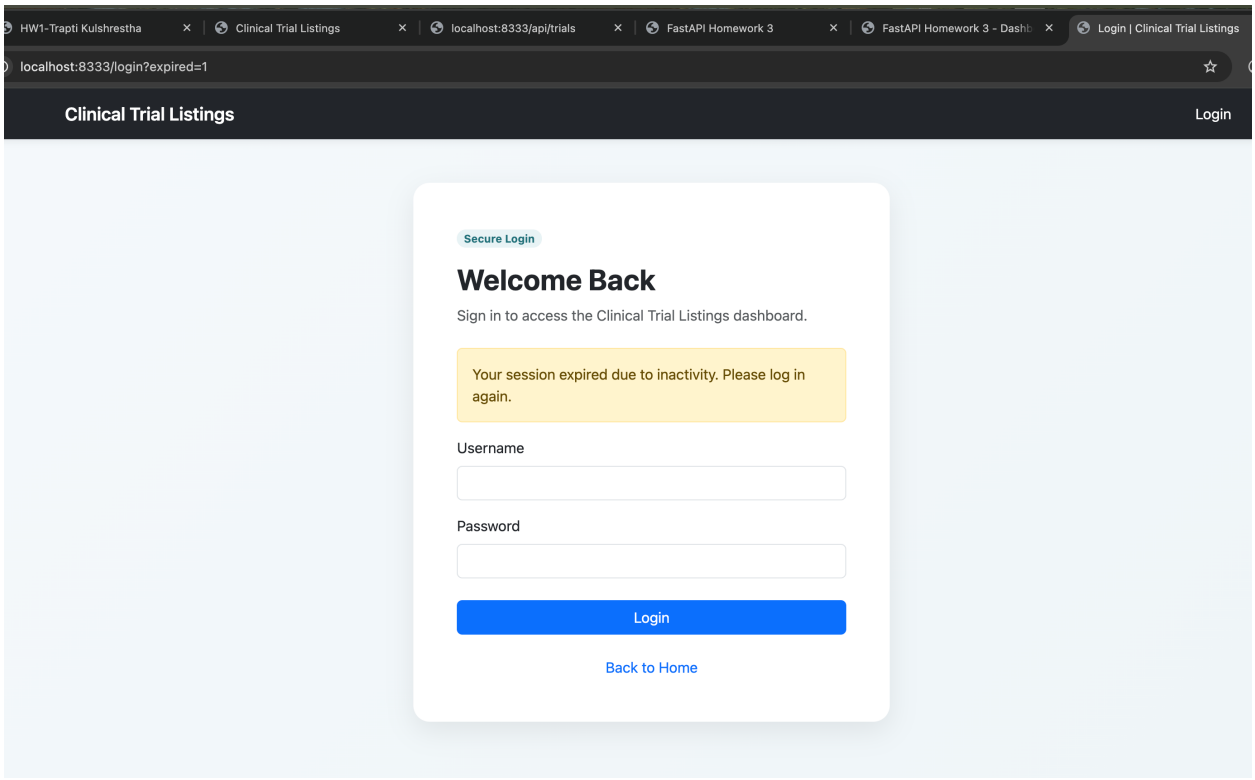


Figure: Login page displayed after the authenticated session expired due to more than 60 seconds of inactivity.

PART 2: LlamaIndex Retrieval Comparison

2.1 Corpus:

I created a local retrieval corpus using three ClinicalTrials.gov study records from different clinical areas: breast cancer, type 2 diabetes, and heart failure. The files were saved locally under data/hw03/ so that all retrieval experiments use the same fixed corpus. The three PDFs together are well above the required minimum corpus size of 200 KB.

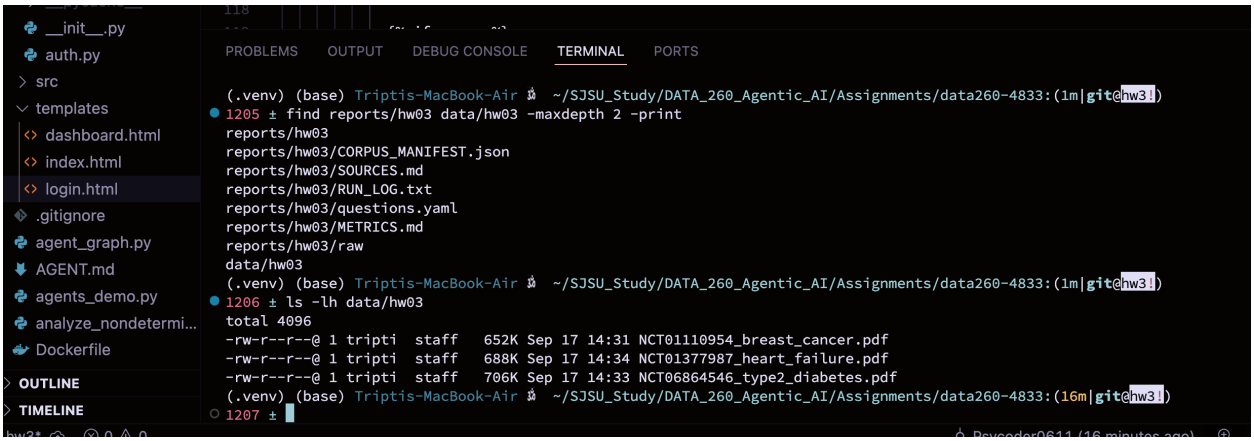


Figure: Local HW3 corpus containing three ClinicalTrials.gov study PDFs and their file sizes.

I generated CORPUS_MANIFEST.json programmatically so the file sizes and SHA256 hashes were recorded exactly. The final corpus contains three PDF files with a combined size of 2,094,862 bytes, which is well above the minimum 200 KB requirement.



Figure: Generated corpus manifest showing the three local ClinicalTrials.gov files, exact byte sizes, total corpus size, and SHA256 hashes.

Before running the retrieval experiments, I verified that text could be successfully extracted from all three local PDF snapshots. The script detected all three corpus files and extracted a total of 32,692 characters, confirming that the documents were readable and suitable for chunking.

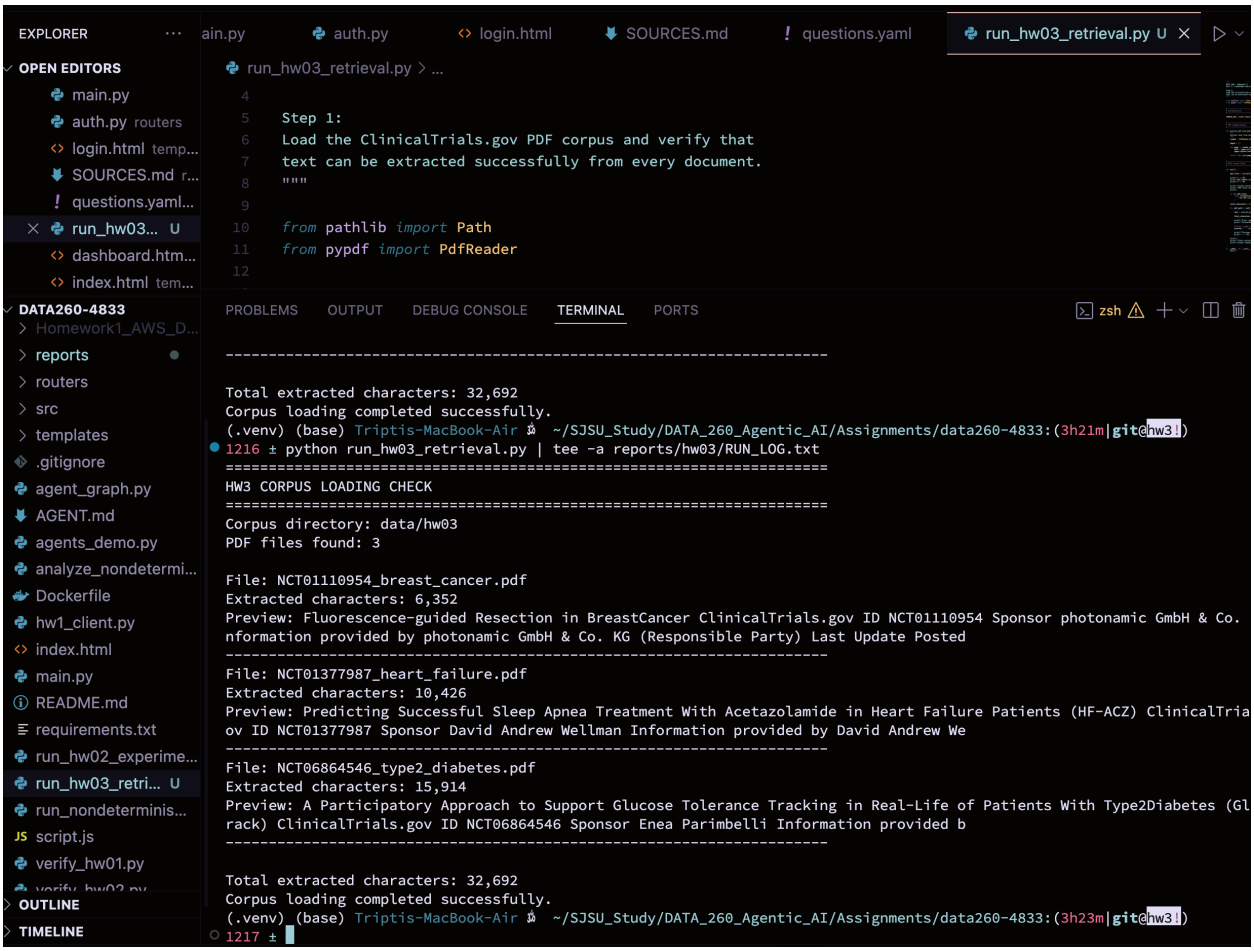


Figure 2.3: Corpus loading sanity check showing successful text extraction from all three ClinicalTrials.gov PDFs.

2.2 Questions:

Before running any retrieval experiments, I created a fixed set of five domain-specific questions and saved them in reports/hw03/questions.yaml. Each question includes the expected answer and the expected source document. I intentionally created questions that cover all three ClinicalTrials.gov study records so the retrieval methods can be compared on the same test set. At least two of the questions depend on information from only one specific source document, which helps evaluate whether the retriever can identify the correct study rather than simply returning a generally similar clinical-trial passage.

I committed questions.yaml to Git before generating retrieval outputs so that the expected answers and source labels would remain fixed and would not be adjusted after seeing the retrieval results.

ID	Questions	Expected Source
Q1	Why was study NCT01110954 terminated?	NCT01110954_breast_cancer.pdf
Q2	What intervention is evaluated in NCT01377987?	NCT01377987_heart_failure.pdf
Q3	What is the recruitment status of NCT06864546?	NCT06864546_type2_diabetes.pdf
Q4	Which study focuses on real-life glucose tolerance tracking?	NCT06864546_type2_diabetes.pdf
Q5	Which study involves sleep apnea treatment in heart failure patients?	NCT01377987_heart_failure.pdf

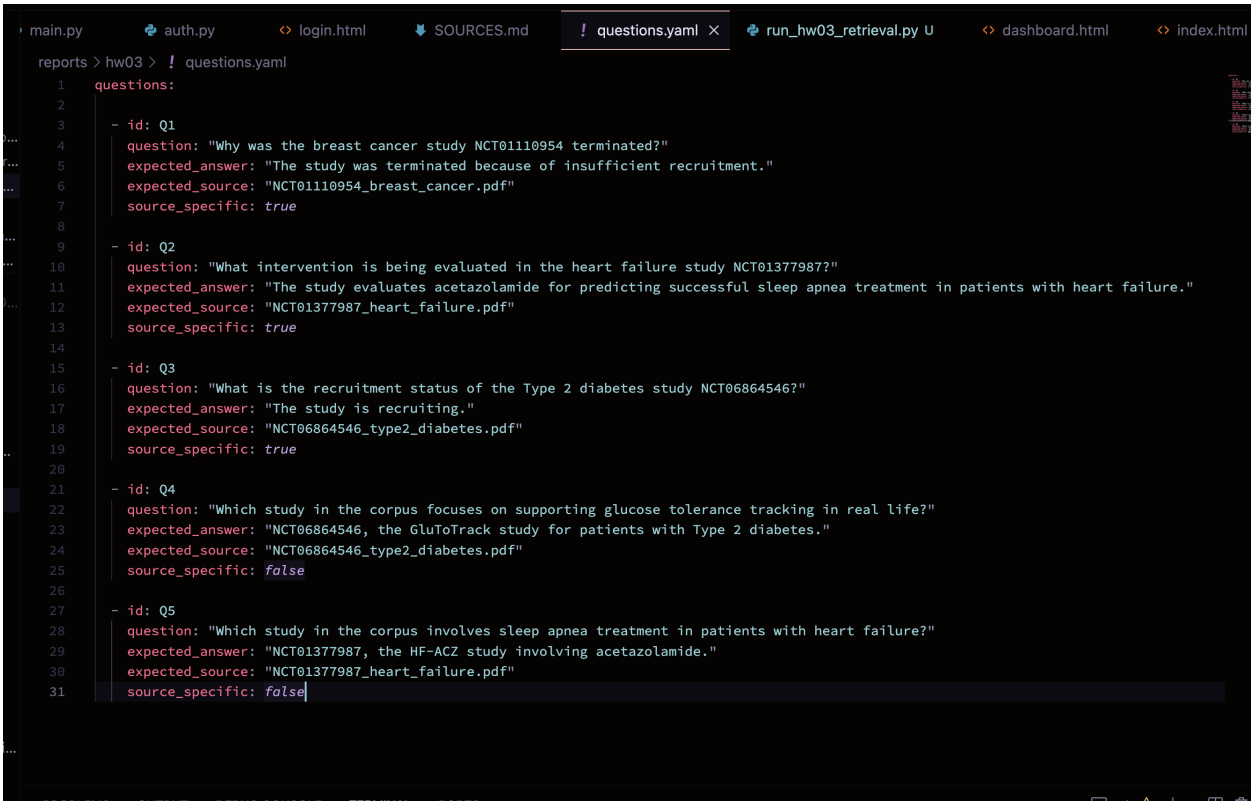


Figure: Predefined retrieval questions with expected answers and source documents, created before running the retrieval experiments.

2.3 Token Chunking and Retrieval:

For the first retrieval configuration, I used LlamaIndex TokenTextSplitter with a chunk size of 512 tokens and an overlap of 50 tokens. The three ClinicalTrials.gov documents were converted into LlamaIndex Document objects and then split into token-based chunks. This produced 19 chunks with an average chunk length of about 1,875 characters. I also verified

that source metadata was preserved, allowing each chunk to be traced back to its original PDF.



The first generated chunk came from the breast cancer study and contained the study title, ClinicalTrials.gov ID, sponsor information, recruitment status, and other study details. This confirmed that the token-based chunking was preserving meaningful clinical trial context.

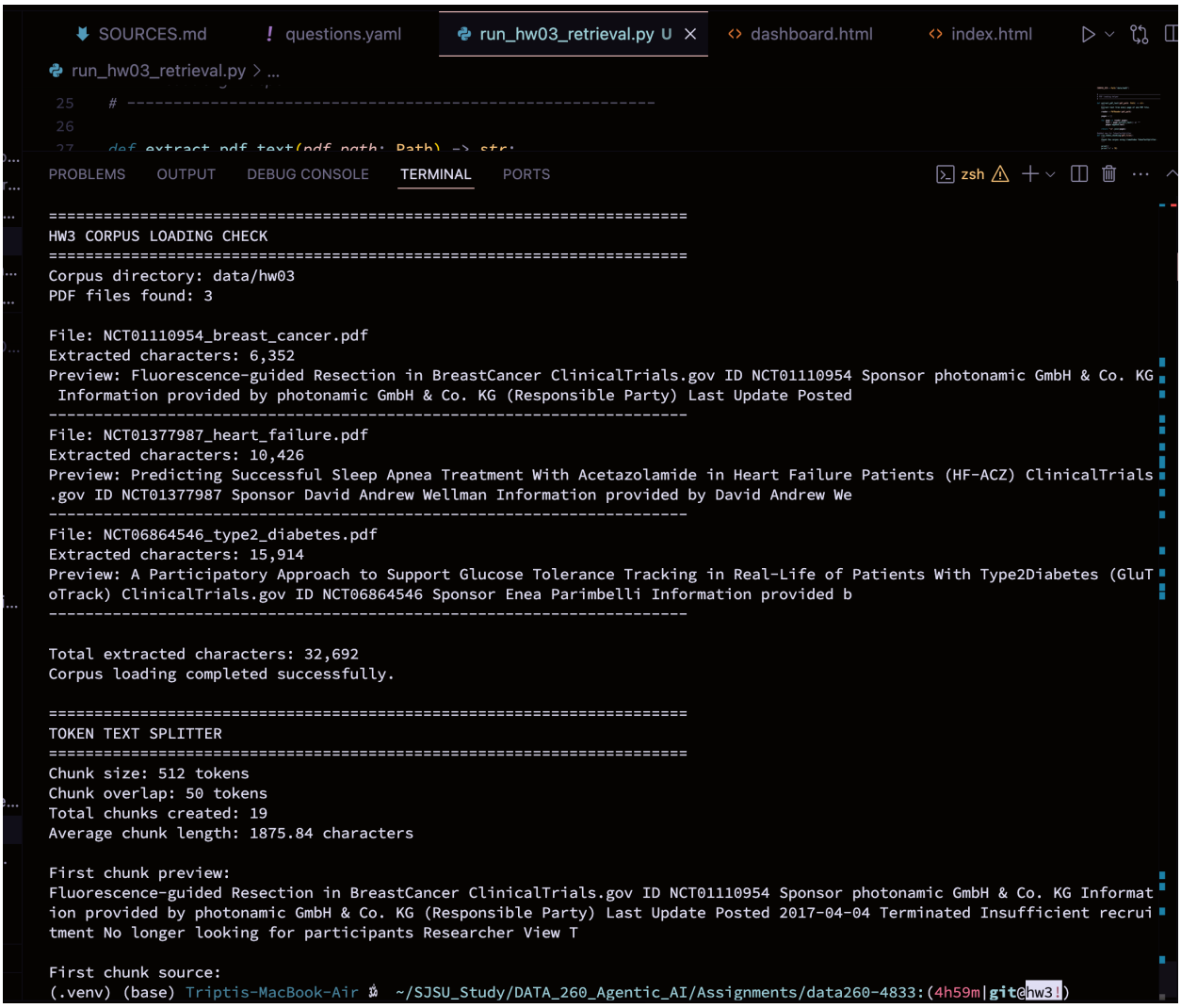
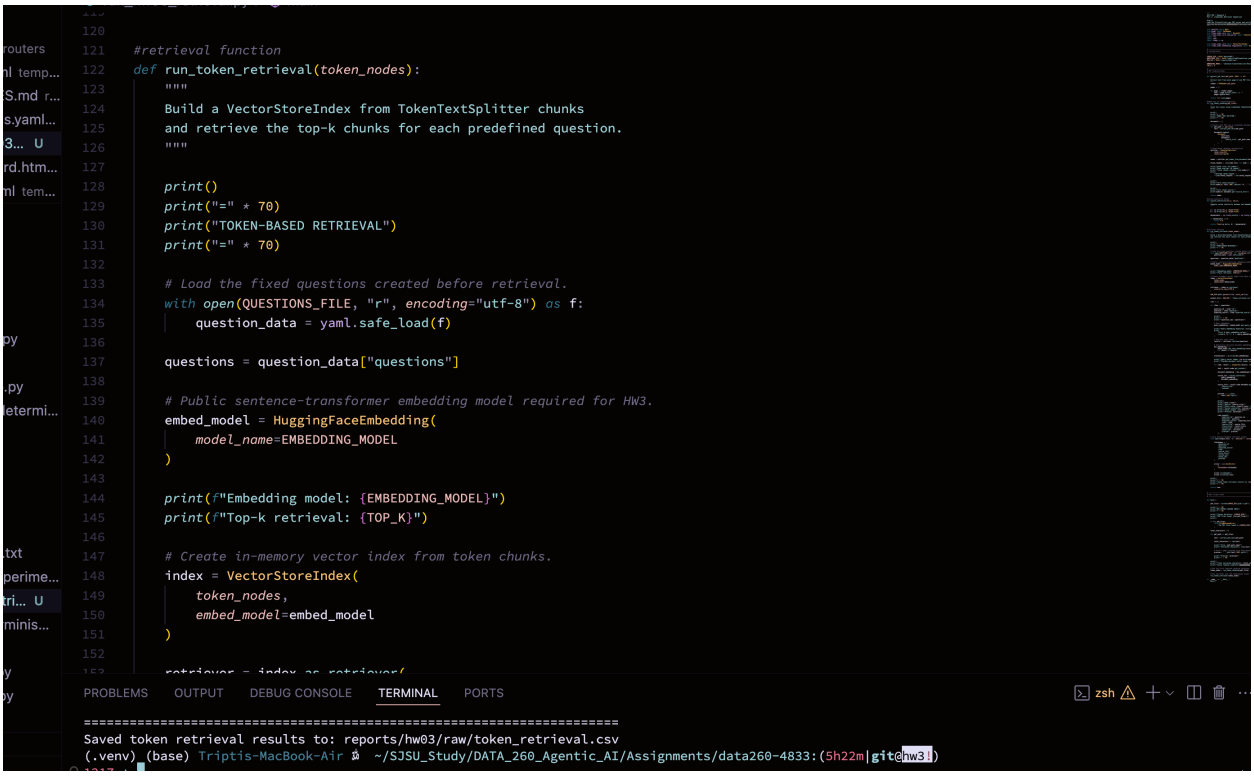


Figure: TokenTextSplitter output showing the 512-token chunk size, 50-token overlap, number of chunks created, average chunk length, preview, and source file.



```python
#retrieval function
def run_token_retrieval(token_nodes):
    """
    Build a VectorStoreIndex from TokenTextSplitter chunks
    and retrieve the top-k chunks for each predefined question.
    """

    print()
    print("=" * 70)
    print("TOKEN-BASED RETRIEVAL")
    print("=" * 70)

    # Load the fixed questions created before retrieval.
    with open(QUESTIONS_FILE, "r", encoding="utf-8") as f:
        question_data = yaml.safe_load(f)

    questions = question_data["questions"]

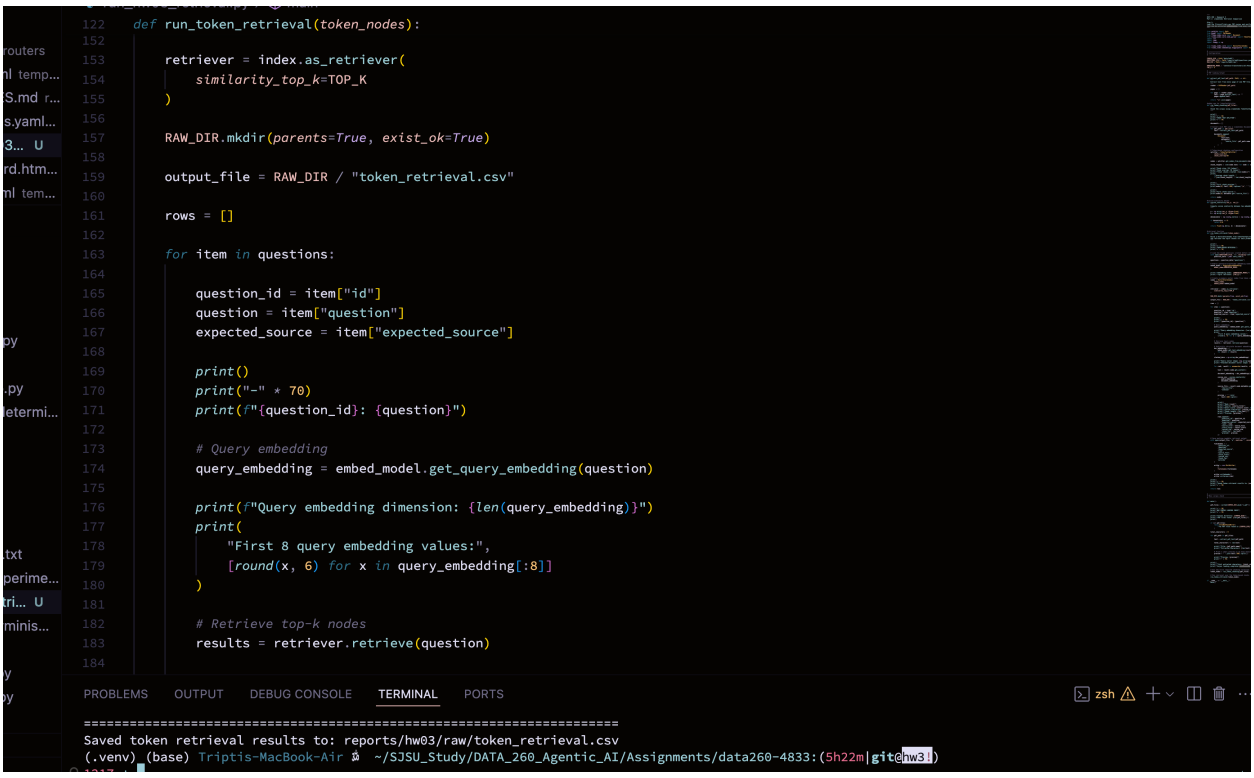
    # Public sentence-transformer embedding model required for HW3.
    embed_model = HuggingFaceEmbedding(
        model_name=EMBEDDING_MODEL
    )

    print(f"Embedding model: {EMBEDDING_MODEL}")
    print(f"Top-k retrieval: {TOP_K}")

    # Create in-memory vector index from token chunks.
    index = VectorStoreIndex(
        token_nodes,
        embed_model=embed_model
    )
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

======================================================================
Saved token retrieval results to: reports/hw03/raw/token_retrieval.csv
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5h22m|git@hw3!)
```



```python
def run_token_retrieval(token_nodes):

    retriever = index.as_retriever(
        similarity_top_k=TOP_K
    )

    RAW_DIR.mkdir(parents=True, exist_ok=True)

    output_file = RAW_DIR / "token_retrieval.csv"

    rows = []

    for item in questions:

        question_id = item["id"]
        question = item["question"]
        expected_source = item["expected_source"]

        print()
        print("-" * 70)
        print(f"{question_id}: {question}")

        # Query embedding
        query_embedding = embed_model.get_query_embedding(question)

        print(f"Query embedding dimension: {len(query_embedding)}")
        print(
            "First 8 query embedding values:",
            [round(x, 6) for x in query_embedding[:8]]
        )

        # Retrieve top-k nodes
        results = retriever.retrieve(question)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

======================================================================
Saved token retrieval results to: reports/hw03/raw/token_retrieval.csv
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5h22m|git@hw3!)
```

After creating the token-based chunks, I built an in-memory VectorStoreIndex using the sentence-transformers/all-MiniLM-L6-v2 embedding model. Each query produced a 384-dimensional embedding, and I retrieved the top three chunks for every predefined

question. For each result, I recorded the vector-store score, an explicitly calculated cosine similarity, chunk length, source file, and a short text preview. The detailed machine-readable results were saved to reports/hw03/raw/token_retrieval.csv.

The token-based retriever identified the correct source document for all five questions. For example, the breast-cancer termination question retrieved only chunks from NCT01110954_breast_cancer.pdf, while the diabetes and heart-failure questions retrieved chunks from their corresponding study files. The top cosine similarities were generally in the range of roughly 0.55–0.64, showing meaningful semantic alignment between the questions and the retrieved chunks.

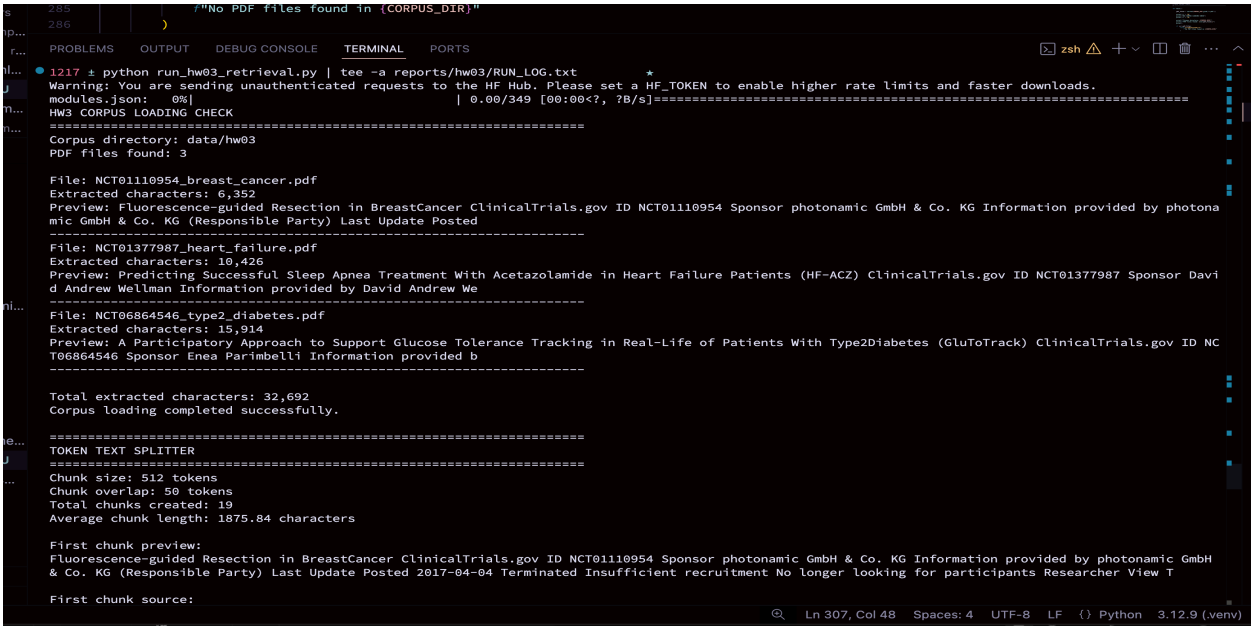


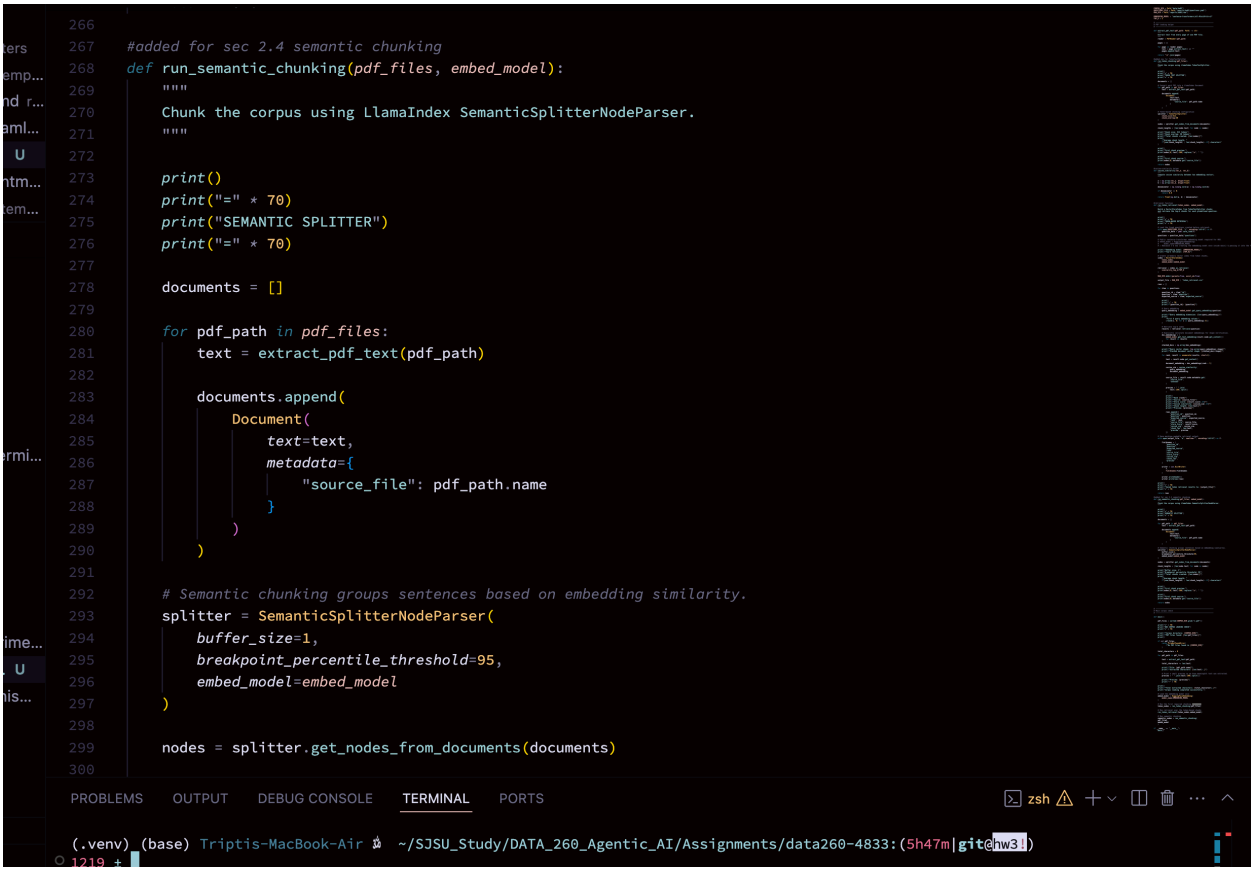




Figure: Token-based top-3 retrieval output showing query embedding details, vector shapes, similarity scores, chunk lengths, and retrieved source documents.

2.4 Semantic Chunking:

I used SemanticSplitterNodeParser for the second chunking strategy. Unlike fixed token chunking, semantic chunking uses the embedding model to identify changes in meaning between neighboring sentences and places chunk boundaries where the semantic difference becomes large. I used a buffer size of 1 and a breakpoint percentile threshold of 95 so that semantically related sentences remain grouped together.



```
#added for sec 2.4 semantic chunking
def run_semantic_chunking(pdf_files, embed_model):
    """
    Chunk the corpus using LlamaIndex SemanticSplitterNodeParser.
    """

    print()
    print("=" * 70)
    print("SEMANTIC SPLITTER")
    print("=" * 70)

    documents = []

    for pdf_path in pdf_files:
        text = extract_pdf_text(pdf_path)

        documents.append(
            Document(
                text=text,
                metadata={
                    "source_file": pdf_path.name
                }
            )
        )

    # Semantic chunking groups sentences based on embedding similarity.
    splitter = SemanticSplitterNodeParser(
        buffer_size=1,
        breakpoint_percentile_threshold=95,
        embed_model=embed_model
    )

    nodes = splitter.get_nodes_from_documents(documents)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
zsh

(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(5h47m|git@hw3!)
1219 +
```

The semantic splitter produced 10 chunks with an average chunk length of approximately 3,269 characters. This is fewer than the 19 chunks created by TokenTextSplitter, which shows that the semantic method grouped more related text together into larger chunks. The source-file metadata was also preserved, so each semantic chunk can still be traced back to the original ClinicalTrials.gov PDF.

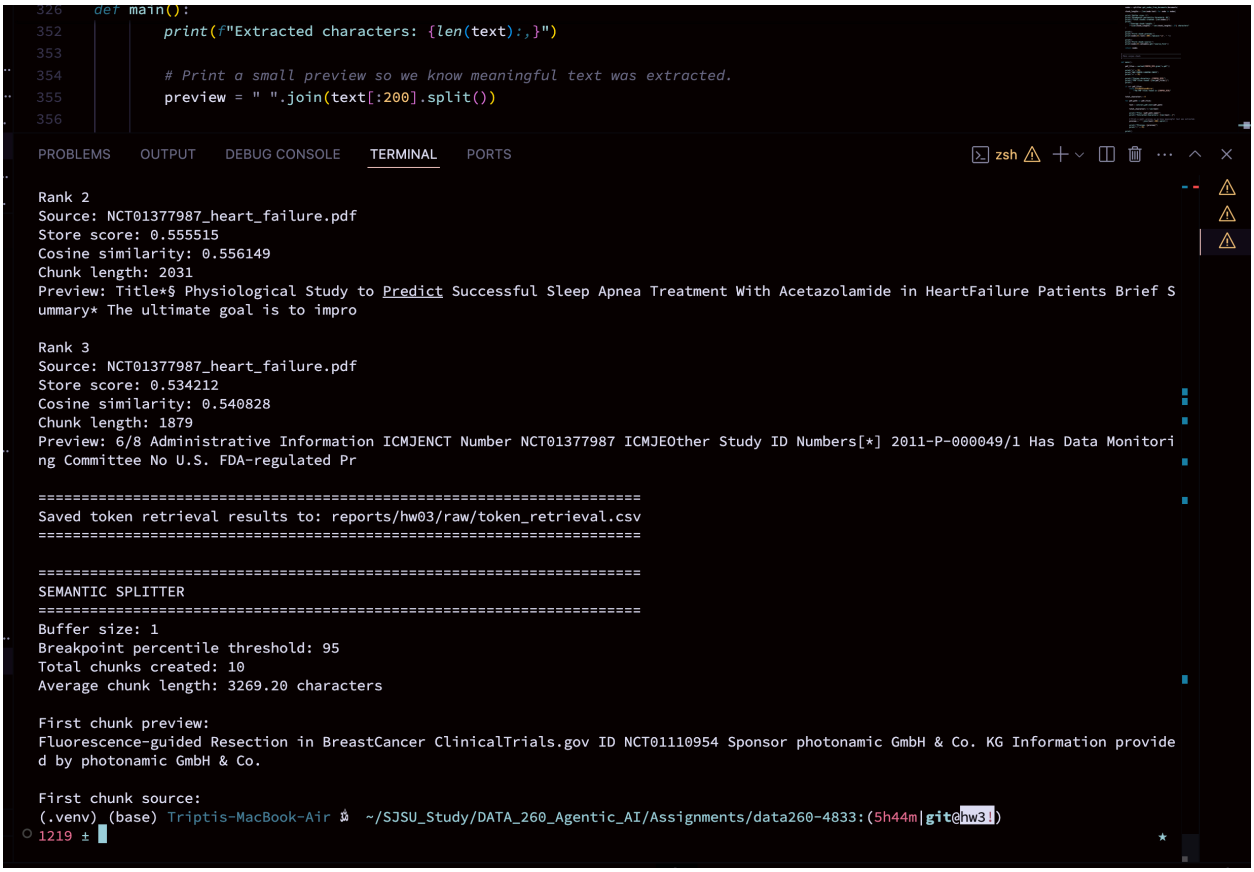


Figure: SemanticSplitterNodeParser output showing buffer size, breakpoint threshold, total chunks, average chunk length, preview, and source file.

Compared with token chunking, semantic chunking created fewer but larger chunks, which may preserve broader context but can also make individual retrieved chunks less focused.

I then used the semantic chunks to perform top-3 retrieval for the same five predefined questions. The same all-MiniLM-L6-v2 embedding model was used so that the retrieval results could be compared fairly with the token-based method. For each query, I recorded the embedding dimension, the first eight embedding values, vector shapes, vector-store score, explicit cosine similarity, chunk length, source file, and a short preview. The results were saved to reports/hw03/raw/semantic_retrieval.csv.

Semantic retrieval correctly identified the expected source document for most questions. However, some retrieved chunks were very large, and one highly ranked result for the heart-failure intervention question was a generic disclaimer rather than a passage containing the answer. This shows that semantic similarity can sometimes rank text that is related to the document context but does not directly answer the question.

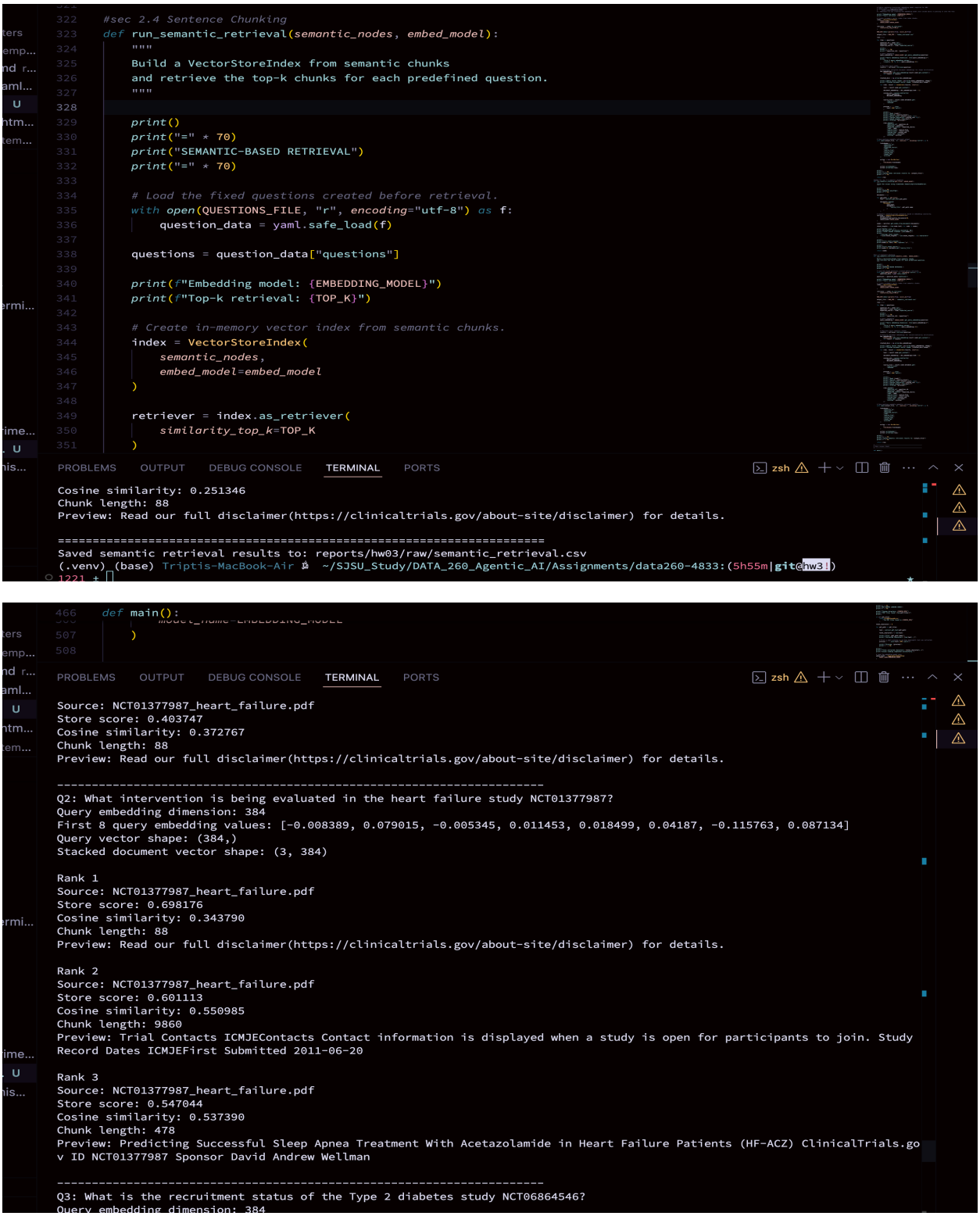
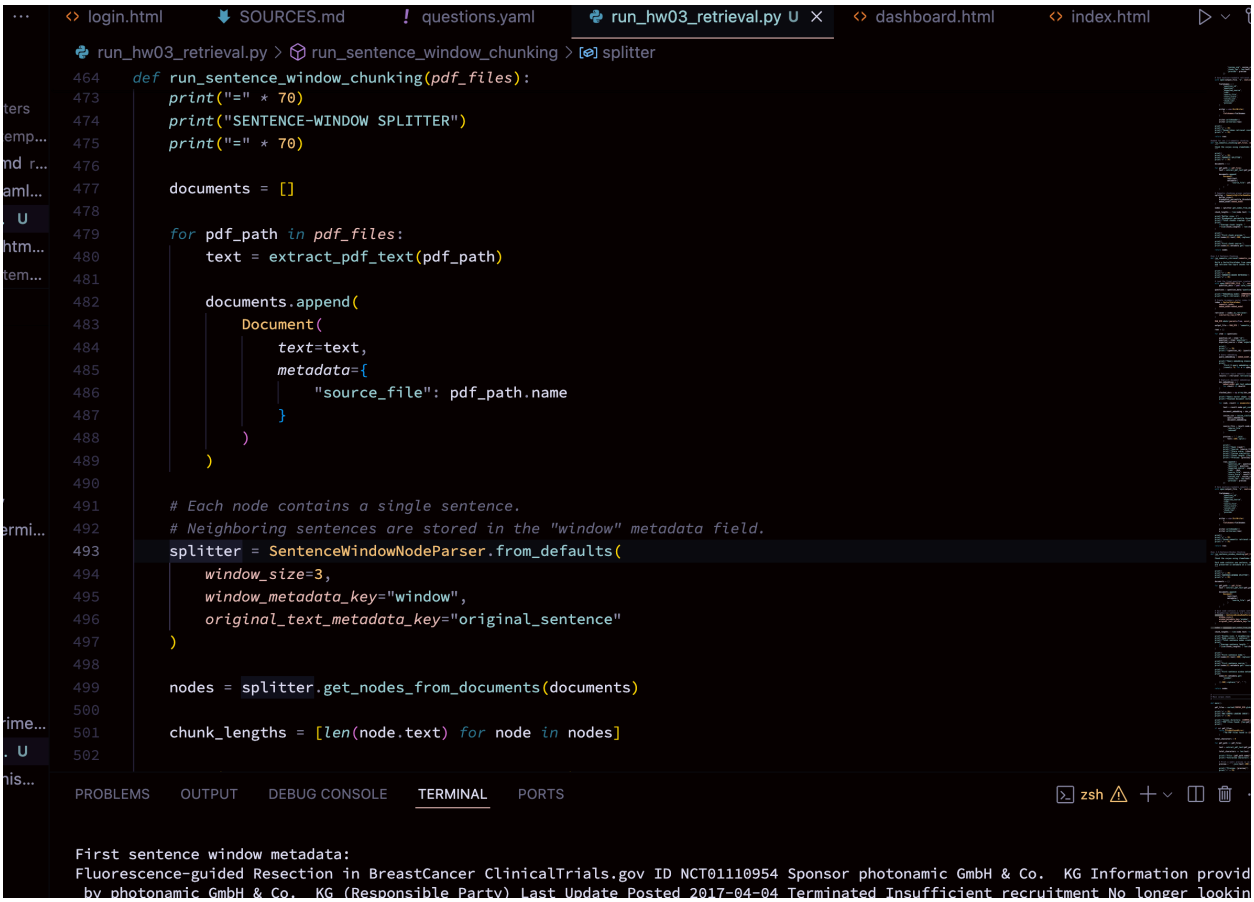


Figure 2.6: Semantic retrieval output showing top-3 results for the heart-failure intervention question, including a high-ranked but non-answering disclaimer chunk.

2.5 Sentence-Window Chunking:

I used SentenceWindowNodeParser as the third chunking strategy. In this method, each indexed node contains a single sentence, while nearby sentences are stored in metadata as a context window. I used a window size of 3 so that each retrieved sentence could still be associated with surrounding context without making the indexed node itself too large.

This method created 115 sentence-level nodes with an average sentence length of approximately 284 characters. This is much more fine-grained than the token and semantic chunking approaches. The output also confirmed that the neighboring sentence window was stored successfully in metadata and that each node retained its original source filename.



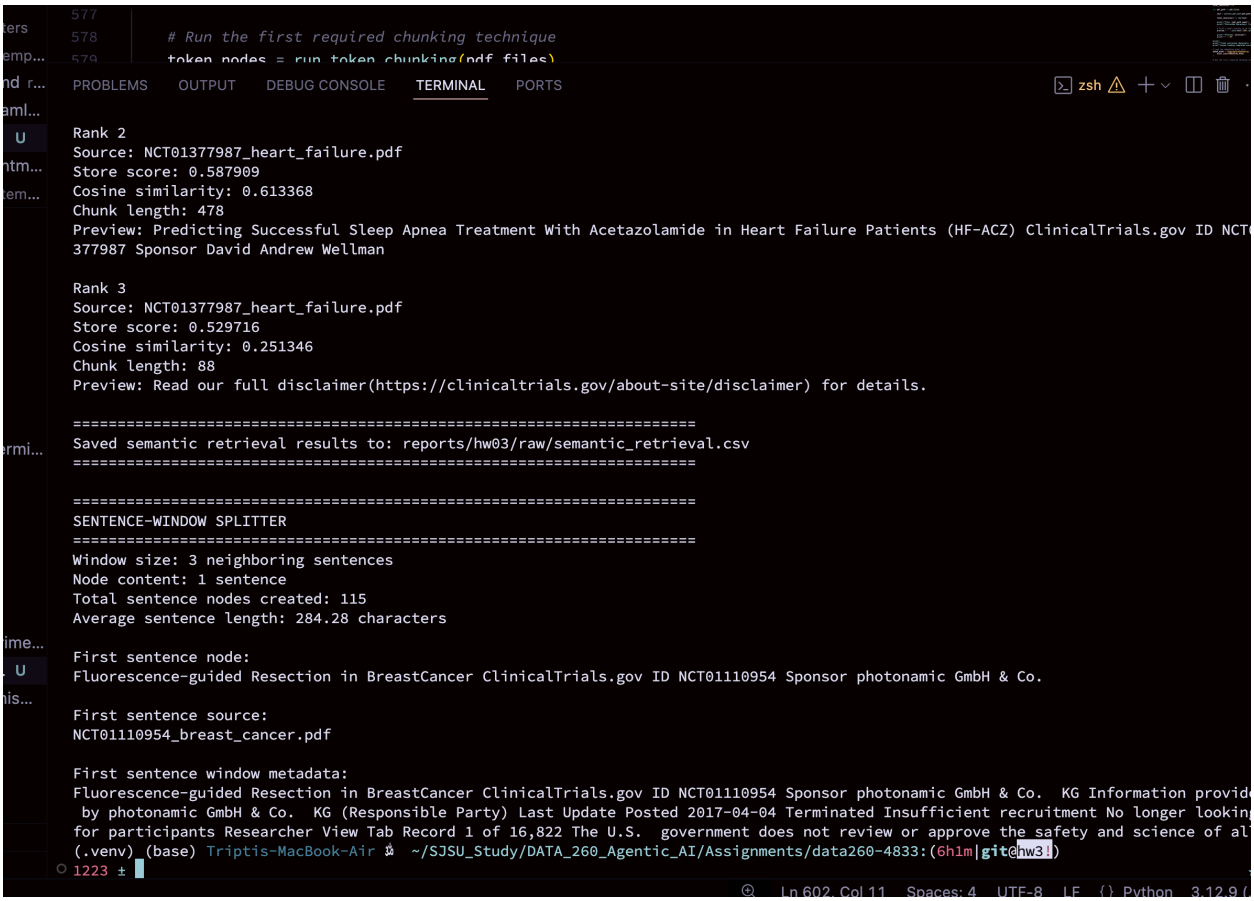


Figure: SentenceWindowNodeParser output showing one-sentence nodes, three-sentence context windows, total node count, average sentence length, and preserved source metadata.

I then performed top-3 retrieval using the sentence-window nodes for the same five predefined questions. The indexed node remained a single sentence, while the surrounding context was obtained from the stored window metadata. For each result, I recorded the source file, vector-store score, explicit cosine similarity, sentence length, window length, and a short preview of the context window.

Sentence-window retrieval produced more fine-grained matches than the other two methods because retrieval was performed at sentence level. The surrounding window metadata helped recover broader context around the matched sentence. For the five questions, the expected study was consistently retrieved among the top results, while the retrieved sentence lengths remained much smaller than the larger semantic chunks.

```python
#sentence-window retrieval
def run_sentence_window_retrieval(sentence_nodes, embed_model):
    """
    Build a VectorStoreIndex from sentence-window nodes
    and retrieve the top-k results for each predefined question.
    """

    print()
    print("=" * 70)
    print("SENTENCE-WINDOW RETRIEVAL")
    print("=" * 70)

    with open(QUESTIONS_FILE, "r", encoding="utf-8") as f:
        question_data = yaml.safe_load(f)

    questions = question_data["questions"]

    print(f"Embedding model: {EMBEDDING_MODEL}")
    print(f"Top-k retrieval: {TOP_K}")

    index = VectorStoreIndex(
        sentence_nodes,
        embed_model=embed_model
    )

    retriever = index.as_retriever(
        similarity_top_k=TOP_K
    )

    RAW_DIR.mkdir(parents=True, exist_ok=True)

    output_file = RAW_DIR / "sentence_window_retrieval.csv"

    rows = []
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh

```
ng medical informatics and medical device w
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh

```
Sentence length: 100
Window length: 1447
Window preview: Predicting Successful Sleep Apnea Treatment With Acetazolamide in Heart Failure Patients (HF-ACZ) ClinicalTrials.gov
ID NCT01377987 Sponsor David Andrew Wellman

----------------------------------------------------------------------
Q3: What is the recruitment status of the Type 2 diabetes study NCT06864546?
Query embedding dimension: 384
First 8 query embedding values: [-0.0272, 0.017865, -0.045162, 0.069113, -0.028147, 0.002071, -0.041797, 0.071156]
Query vector shape: (384,)
Stacked document vector shape: (3, 384)

Rank 1
Source: NCT06864546_type2_diabetes.pdf
Store score: 0.718565
Cosine similarity: 0.711893
Sentence length: 924
Window length: 2387
Window preview: Diabetologia. 2017 Mar;60(3):490- 498. doi: 10.1007/s00125-016-4161-7. Epub 2016 Nov 30. (https://pubmed.ncbi.nlm.ni
.gov/27904925) Recruitment Information IC

Rank 2
Source: NCT06864546_type2_diabetes.pdf
Store score: 0.712285
Cosine similarity: 0.651721
Sentence length: 531
Window length: 755
Window preview: A Participatory Approach to Support Glucose Tolerance Tracking in Real-Life of Patients With Type2Diabetes (GluToTra
k) ClinicalTrials.gov ID NCT06864546 Sponso

Rank 3
Source: NCT06864546_type2_diabetes.pdf
Store score: 0.710104
Cosine similarity: 0.382144
Sentence length: 28
Window length: 510
Window preview: Type 2 Diabetes and the Use of Real-Time Continuous Glucose Monitoring. Diabetes Technol Ther. 2021 Mar;23(S1):S27-S
4. doi: 10.1089/dia.2021.0007. (https:/

----------------------------------------------------------------------
Q4: Which study in the corpus focuses on supporting glucose tolerance tracking in real life?
Query embedding dimension: 384
First 8 query embedding values: [-0.015759, -0.007772, -0.099241, 0.046852, 0.048499, 0.022574, 0.050065, 0.055823]
Query vector shape: (384,)
```

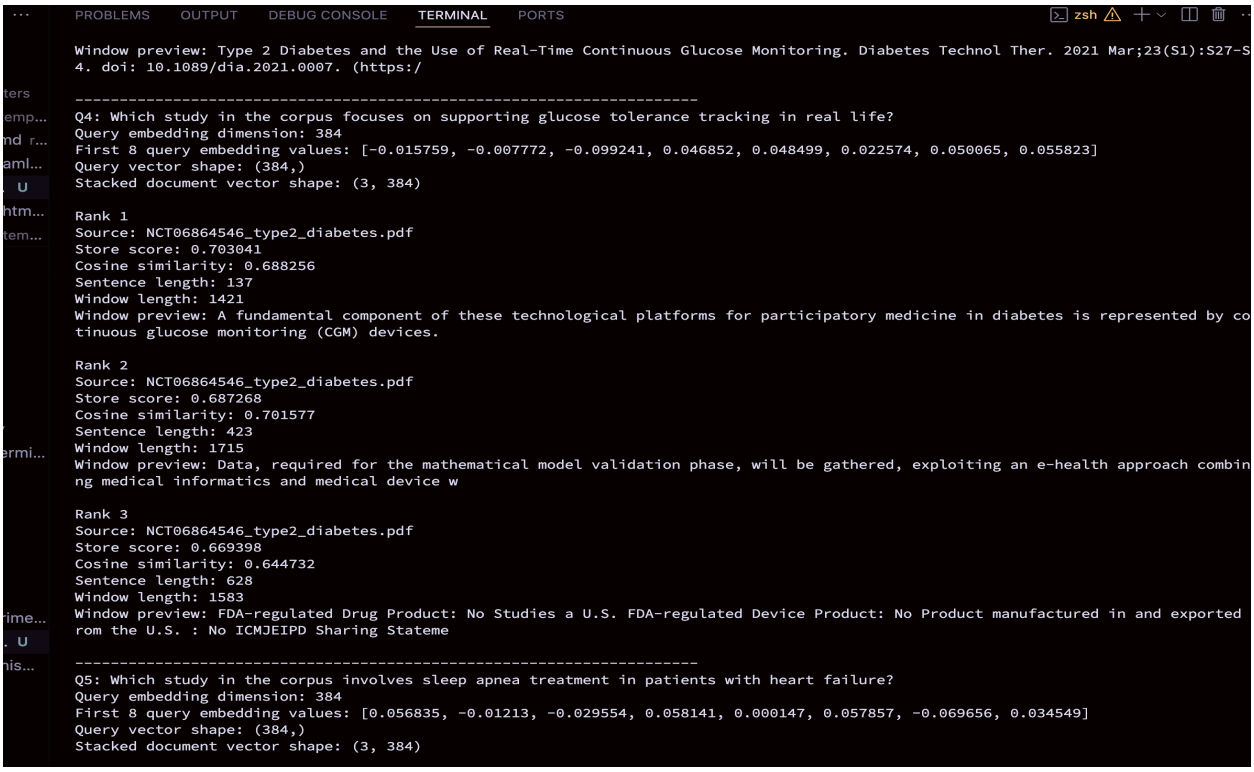


Figure: Sentence-window retrieval output showing sentence-level matches together with surrounding context-window metadata.

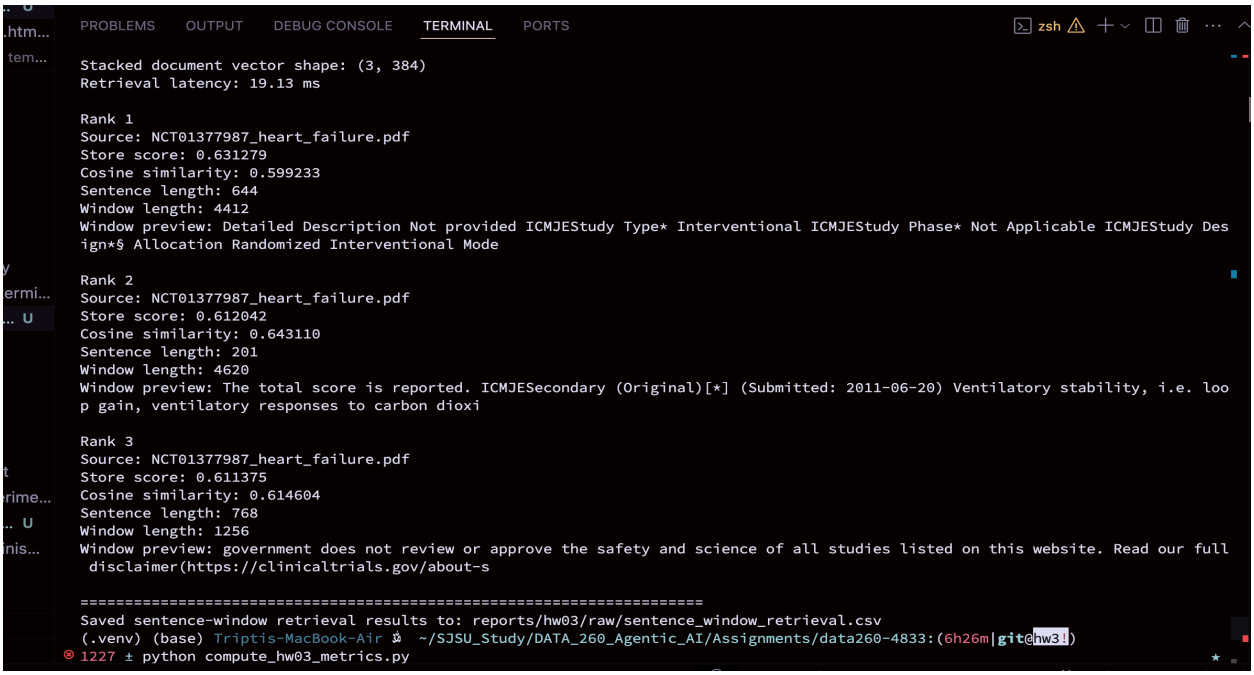


Figure 2.8: Sentence-window retrieval output showing top-3 results, cosine similarities, sentence/window lengths, and retrieved source metadata for the heart-failure query.

2.6 Retrieval Metrics:

compared all three retrieval methods using the same five questions & the same sentence-transformers/all-MiniLM-L6-v2 embedding model. For each method, I looked at the Top-1 cosine similarity, Mean@3 cosine similarity, total number of chunks, average chunk length, retrieval latency & Recall@3.

Token chunking gave the highest Top-1 cosine score (0.5910) & the highest Mean@3 score (0.5611). Semantic chunking created the fewest chunks (10), but those chunks were much larger, with an average length of 3269.20 characters. Sentence-window chunking created the most nodes (115) & had the smallest average length (284.28 characters). In the final run, the average retrieval latency was 8.16 ms for Token, 10.08 ms for Semantic & 10.01 ms for Sentence-Window. All three methods achieved Recall@3 = 1.00, so the expected source appeared in the top three results for every question.

Looking at the results together, all three methods were able to find the correct source, but they handled the text differently. Token chunking gave a good balance between similarity score, chunk size & speed. Semantic chunking kept more context together in fewer large chunks, while sentence-window retrieval worked with much smaller sentence-level pieces & used nearby window text for extra context.

Figure: Comparison of retrieval metrics for Token, Semantic, and Sentence-Window chunking.

Technique	Top-1 Cosine	Mean@3	Chunks	Avg Length	Latency (ms)	Recall@3
Token	0.5910	0.5611	19	1875.84	8.16	1.00
Semantic	0.5683	0.5330	10	3269.20	10.08	1.00
Sentence-Window	0.5267	0.5008	115	284.28	10.01	1.00

2.7 Wrong but Confident Retrieval:

During the semantic retrieval experiment, I found a confidently ranked result that did not actually contain the answer to the question. For Q2, which asked about the intervention evaluated in the heart-failure study NCT01377987, the Rank 1 result was a generic ClinicalTrials.gov disclaimer. Even though this chunk did not answer the question, it received a relatively high vector-store score of 0.698176.

This happened because embedding-based retrieval measures semantic similarity rather than whether a passage literally contains the correct answer. A chunk can be considered similar because it belongs to the same study page or shares related contextual language,

even when it does not include the specific information being requested. This example shows why a high similarity score should not automatically be interpreted as a correct answer.

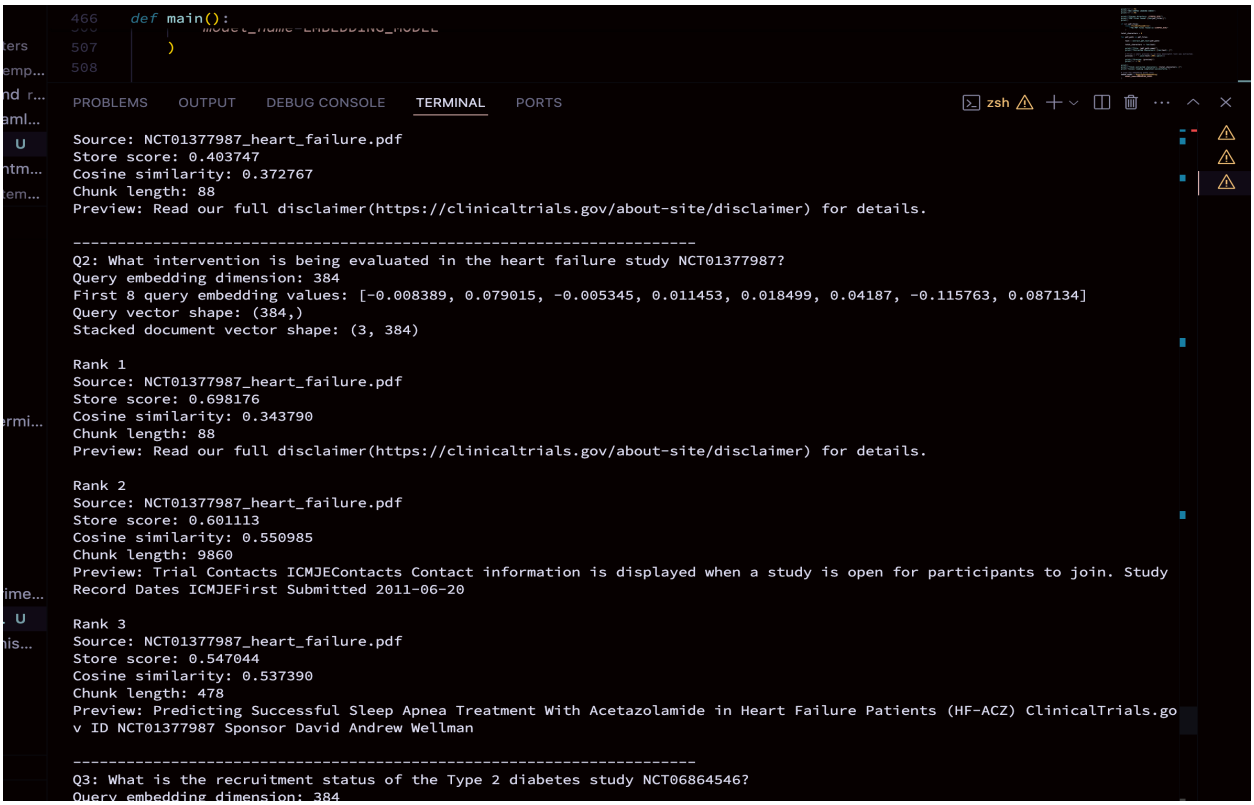


Figure: Example of a high-ranked semantic retrieval result that is contextually related to the study but does not answer the intervention question.

2.8 Comparison and Conclusion:

The three chunking approaches produced noticeably different retrieval behavior. Token chunking created medium-sized overlapping chunks and achieved the highest average Top-1 and Mean@3 cosine similarities in this experiment. Semantic chunking created fewer but much larger chunks, which preserved broader context but sometimes retrieved overly broad or irrelevant content, such as the high-ranked disclaimer example. Sentence-window chunking created many small sentence-level nodes and used neighboring sentences as metadata to restore context around each result.

Overall, all three methods achieved *Recall@3 = 1.00*, meaning the expected source appeared within the top three results for every predefined question. For this small Clinical Trial Listings corpus, token chunking provided a good balance between retrieval quality, chunk size, and speed, while the other two methods highlighted useful trade-offs between context preservation and retrieval precision.

AI Use:

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT and Claude mainly for guidance while extending my existing FastAPI project, understanding the authentication/session requirements & the LlamaIndex retrieval pipeline & debugging errors as they appeared.

I personally selected the three ClinicalTrials.gov studies, created the local corpus, ran all commands & experiments, tested the login/logout/session behavior, captured the screenshots, reviewed the retrieval outputs & verified the generated files & metrics in my local environment.

2. What was one AI-produced suggestion that was unsuitable, or one result that I independently verified?

I independently verified the suggested FastAPI and retrieval code instead of assuming it would work directly. During the authentication setup, the provided starter code used a TemplateResponse calling style that was not compatible with my installed Starlette version. I also verified every retrieval technique by checking the actual terminal output and saved CSV files.

3. How did I detect the problem or verify the result?

I detected the TemplateResponse problem because the application returned a 500 error with TypeError: unhashable type: 'dict'. I used the traceback to identify the failing call in routers/auth.py.

For the retrieval work, I verified the results by printing the embedding dimensions, vector shapes, retrieved source files, store scores, cosine similarities, chunk lengths, previews, and retrieval latency. I also compared the retrieved sources against the expected sources recorded earlier in questions.yaml.

4. What did I change, and why does it work now?

I updated the TemplateResponse calls to use explicit request, name & context arguments, which matched the installed Starlette API & allowed the templates to render correctly.

For retrieval evaluation, I kept the five questions fixed before running the experiments, saved machine-readable outputs for all three techniques, and added explicit cosine-similarity and latency calculations. This made the comparison reproducible & allowed the final metrics to be computed directly from the saved raw results rather than manually.

Verification:

I created a self-check script, verify_hw03.py, to verify that the final HW3 implementation and required artifacts were complete and reproducible. The script checks that the main Python files compile successfully, the local corpus contains the required three PDFs and exceeds 200 KB, questions.yaml contains the five precommitted questions, all three raw retrieval CSV files contain the expected rows and columns, Recall@3 is 1.00 for all three techniques, and the required HW3 documentation files are present.

The verification completed successfully with an overall PASS, and the machine-readable results were saved to reports/hw03/verification.json.

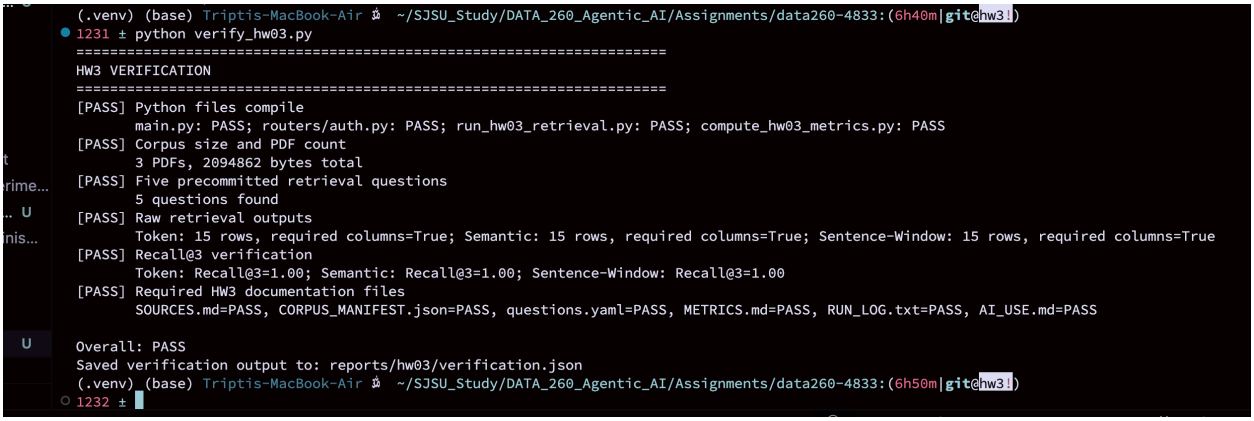


Figure: HW3 self-check output showing all verification checks passing and verification.json being generated.

Final GitHub Information:

This homework was completed in the existing data260-4833 repository on the hw3 branch. The final submitted repository state will be tagged as hw3. After completing all code, report, metrics, verification, and documentation updates, I will record the final tagged commit hash and repository link below so that the submitted report points to the exact version used for grading.

Appendix A: auth.py

main.py verify_hw03.py README.md hw1_client.py auth.py login.html

iters > auth.py > ...

```python
from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND
import time


# Create a router object
# This behaves like a mini FastAPI app
router = APIRouter()

# Configure Jinja2 templates directory
templates = Jinja2Templates(directory="templates")


# Hardcoded credentials for demo purposes only
# In real applications, credentials come from a database
VALID_USERNAME = "admin"
VALID_PASSWORD = "password"

# Idle timeout used for HW3 demonstration.
# If the user is inactive for more than 60 seconds,
# the session is cleared.
IDLE_TIMEOUT_SECONDS = 60

```

```python
@router.get("/")
def home(request: Request):
    """
    Home page route.

    - Checks if a user is logged in using the session
    - Passes user info to the template
    """
    user = request.session.get("user")

    return templates.TemplateResponse(
    request=request,
    name="index.html",
    context={
        "user": user
    }
)


@router.get("/login")
def login_page(request: Request):

    user = request.session.get("user")

    expired = request.query_params.get("expired")

    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": user,
            "expired": expired
        }
    )
```

```python
@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        # Store logged-in user in session
        request.session["user"] = username

        request.session["last_activity"] = time.time() #added for timelimit

        # Redirect user to dashboard
        return RedirectResponse(
            url="/dashboard",
            status_code=HTTP_302_FOUND
        )

    # If credentials are invalid:
    # Redirect back to login page
    #
    # NOTE:
    # No error message is shown intentionally.
    # Students are expected to add Bootstrap alerts.
    return templates.TemplateResponse(  #replaced this section...hw3
    request=request,
    name="login.html",
    context={
        "user": None,
        "error": "Invalid username or password."
    },
    status_code=401
)
```

```python
@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected dashboard route.

    The user must have a valid session.
    The session also expires after a period of inactivity.
    """

    user = request.session.get("user")
    last_activity = request.session.get("last_activity")

    # No authenticated user -> redirect to login
    if not user:
        return RedirectResponse(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # Check whether the session has been idle too long
    if last_activity is not None:

        idle_time = time.time() - last_activity

        if idle_time > IDLE_TIMEOUT_SECONDS:

            # Remove all session data after timeout
            request.session.clear()

            return RedirectResponse(
                url="/login?expired=1",
                status_code=HTTP_302_FOUND
            )
```

```python
    # User is still active, so refresh the last activity time
    request.session["last_activity"] = time.time()

    return templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={
            "user": user
        }
    )
```

Appendix B: Template Directory

